

THE UNIVERSITY OF QUEENSLAND
AUSTRALIA

Ransomware.agent: AI-powered Webscraping for Textual Data Extraction from Ransomware Data Leak Sites

by

Kintaro Kawai

School of Electrical Engineering and Computer Science

The University of Queensland.

Submitted for the degree of Bachelor of Engineering (Honors) in the division of Software Engineering as part of the course REIT4841.

Date of Submission: 10/11/2025

Kintaro Kawai
s4698570@uq.edu.au

10/11/2025

Prof Michael Brünig,
Head of School
School of Electrical Engineering and Computer Science
The University of Queensland
St Lucia QLD 4072

Dear Professor Brünig,

In accordance with the requirements of the Degree of Bachelor of Engineering (Honours) in the School of Electrical Engineering and Computer Science, I submit the following thesis entitled

"Ransomware.agent*: AI-powered Webscraping for Textual Data Extraction from Ransomware Data Leak Sites"*

The thesis was performed under the supervision of Mr Omar Jarkas, Mr David Gaul, and Professor Ryan Ko. I declare that the work submitted in the thesis is my own, except as acknowledged in the text and footnotes, and that it has not previously been submitted for a degree at the University of Queensland or any other institution.

Yours sincerely,

Kintaro Kawai

Declaration by Author

This thesis is composed of my original work, and contains no material previously published, written by another person, or generated by artificial intelligence (AI), except where clearly referenced and acknowledged either in the main text or the preliminary pages of this thesis. I have clearly stated the contribution of others to jointly authored works that I have included in my thesis.

I have clearly stated the contribution of others to my thesis as a whole, including statistical assistance, survey design, data analysis, significant technical procedures, professional editorial advice, use of AI tools, financial support and any other original research work used or reported in my thesis. The content of my thesis is the result of work I have carried out since the commencement of my enrolment in the course code indicated on the title page, and does not include a substantial part of work that has been submitted to qualify for the award of any other degree or diploma in any university or other tertiary institution. I have clearly stated which parts of my thesis, if any, have been submitted to qualify for another award.

I acknowledge that copyright of all material contained in my thesis resides with the copyright holder(s) of that material. Where appropriate I have obtained copyright permission from the copyright holder to reproduce material in this thesis and have sought permission from co-authors for any jointly authored works included in the thesis.

Use of AI Statement

I acknowledge the use of generative AI tools in completing this assignment. Details of which tools were used and how they were used are provided in the table below, along with appropriate in-text and full references. I take responsibility for critically evaluating and integrating the AI-generated content, and ensuring it adheres to academic integrity standards.

AI Model and Date	**Artificial Intelligence (AI) Tool Usage** (tick all that apply)								
	language translation	**grammar/style**	**planning/drafting**	**research/background information**	**content creation - text**	**content creation - visual**	**content creation - code**	**feedback**	**other (provide details)**
ChatGPT 5 10/11/25		✓	✓	✓	✓		✓		

Acknowledgments

I am sincerely thankful to my supervisors, Dr Omar Jarkas, Dr David Gaul, and Professor Ryan Ko, for their invaluable guidance and support throughout this project.

Dr Jarkas's technical insight and thoughtful feedback helped sharpen the focus of this research and guided the way it was carried out. Dr Gaul's attention to detail and feedback strengthened the structure and quality of this work. Professor Ko's leadership in cybersecurity, along with his encouragement to explore the real-world applications of this study, inspired the practical development of *Ransomware.agent* as an investigative tool.

I also extend special thanks to Dr Julien Mousqueton for generously sharing his expertise and insights, which greatly deepened my understanding of the Ransomware.live platform. His guidance played an important role in shaping the direction of this project.

I also extend sincere appreciation to Luke and Michael, whose ideas and collaboration during the design and development stages were vital to bringing *Ransomware.agent* to life. Their input on testing and evaluation helped the project reach its goals.

Finally, I am deeply grateful to my family and friends for their constant encouragement, patience, and belief in me throughout this journey. Their support is the foundation that allowed this thesis to come to completion.

Abstract

Ransomware incidents have escalated to over 11,000 daily attacks globally in 2025, representing a 3,500% increase over five years, with average ransom payments reaching $2 million per incident [1][2]. Ransomware monitoring systems provide critical intelligence for threat analysis and incident response by tracking which organisations have been compromised. However, existing collection methods rely on conventional web parsers that require manual development for each data leak site. These parsers break when site structures change. This research develops and evaluates *Ransomware.agent*, an autonomous AI-powered web scraping system that uses large language models to extract ransomware victim data without site-specific programming. The agent implements a Sense-Plan-Execute architecture that interprets page structures, generates navigation strategies, and extracts victim records through natural language reasoning. Performance is evaluated against Ransomware.live, the industry-leading open source aggregation platform, by comparing extracted data using cosine similarity between sentence embeddings across five ransomware groups: Akira, DragonForce, Qilin, Play, and The Gentlemen. Results reveal that coverage ranges from 41.7% to 100%. Aggregate similarity scores range from 60.45% to 75.56%. Field-specific accuracy for victim names and domains consistently achieves 80-100% when information appears in structured formats. The agent successfully handles extensive pagination, demonstrated by 90.5% coverage across Play's 70 pages. However, it encounters systematic limitations with high information density, unconventional interfaces, and operational resilience. Results confirm that AI-powered extraction represents a viable complementary approach to conventional parsing. The approach excels at rapid deployment whilst requiring architectural improvements for production-scale reliability.

Contents

Declaration by Author iii
Use of AI Statement iv
Acknowledgments v
Abstract vi
Contents vii
1 Introduction 9
2 Background 13
2.1 Ransomware 13
2.2 Cryptocurrency 13
2.3 TOR Browser 13
2.4 Ransomware as a service (RaaS) 14
2.5 Data Leak Sites (DLS) 14
2.7 Web Scraping and Parsing HTML 14
2.8 Large Language Models 15
2.8.1 LLMs use for HTML 15
2.8.2 Prompt Engineering 16
2.9 Agentic Workflows 16
3 Literature Review 17
3.1 Ransomware DLS monitoring 17
3.2 Agentic Web Navigation: Agent-E 20
4 Methodology 23
4.1 System Design 23
4.1.1 High Level Architecture 23
4.1.2 Sense 25
4.1.3 Plan 27
4.1.4 Execute 28
4.1.5 Action Library 29
4.2 Prompt Engineering 32
4.2.1 Scrape and Store 32
4.2.2 Planning Context 33

4.2.3 Browser Manager ····· 36
4.3 Evaluation ····· 39
4.3.1 Evaluation Approach ····· 39
4.3.2 Coverage Measurement ····· 39
4.3.3 Vector Similarity Analysis ····· 40
5 Results ····· 42
5.1 Akira: Unique Terminal Interface Design ····· 42
5.2 Qilin: Structured Table Navigation ····· 45
5.3 DragonForce: High Information Density Exceeding Output Token Capacity ····· 47
5.4 The Gentlemen: Navigation Simplicity Without Extraction Quality Guarantee ····· 49
5.5 Play: Connection Stability as the Primary Coverage Constraint ····· 51
6 Conclusion ····· 55
7 Future Work ····· 57
7.1 Addressing Operational Resilience and Content Volume Limitations ····· 57
7.2 Generalising Beyond Data Leak Sites ····· 58
7.3 Improving Evaluation Metrics for Semantic Inference ····· 58
8 Closing Remarks ····· 60
9 References ····· 61
10 Appendix ····· 66

1. Introduction

Ransomware attacks increased by 47% in the first half of 2025 compared to 2024, with groups like LockBit, RansomHub, and Akira collectively responsible for over 40% of the 3,627 documented incidents, while average ransom demands escalated from $400,000 to $2 million between 2023 and 2024 [3][4][1]. These attacks have evolved into a structured and profitable criminal business model in which attackers encrypt or steal victim data and demand payment for its release or deletion [5]. Organisations in sectors such as healthcare, finance, and government are known to be frequent targets as they hold highly valuable personal data and are vulnerable to disruptions that can have wide-reaching consequences [6][7]. Modern ransomware operations rely on cryptocurrencies for anonymous transactions [8] and use "double extortion" tactics in which stolen data is published on Data Leak Sites (DLS) hosted through The Onion Router (TOR) network [9][10]. These leak sites typically publish victim names, ransom negotiations, and stolen materials. Open-source projects such as RansomWatch, Ransomlook, and Ransomware.live help monitor this ecosystem by automatically collecting and sharing information from these sites in near real time [11][12][13].

However, these monitoring systems face a critical limitation. They rely on manually designed parsers built for specific website structures. When a ransomware group changes its site layout, these parsers break, and data collection stops until they are repaired [14][15]. The same issue arises when a new Ransomware as a Service (RaaS) group launches a new DLS. This maintenance challenge highlights the need for more adaptive extraction methods that continue operating when site structures change or new sites are launched.

Recent research has demonstrated that large language models can interpret raw HTML structure with considerable accuracy. Gur et al. (2023) showed that pretrained language models can understand how web pages are structured and generate descriptions of rendered content without requiring fine-tuning, achieving approximately 84% accuracy in description generation tasks [30]. This capability suggests that language models possess the structural understanding necessary to navigate and extract information from websites without requiring manually coded parsers for each unique layout.

This thesis explores whether large language models combined with agentic workflows can improve the consistency and quality of data collection compared with traditional, hand-coded parsers. The research addresses two central questions: ***Can AI-powered web scraping accurately extract textual ransomware intelligence from diverse and frequently changing***

HTML structures? and ***Does the data collected by this approach match the completeness and reliability of conventional, manually maintained parsers?***

This twelve-month honours project develops and tests *Ransomware.agent*, an autonomous AI-powered monitoring tool that uses large language models to scrape ransomware leak sites. The study examines publicly accessible TOR-hosted leak sites that require no authentication. All activities follow university-approved ethical and legal standards for dark web research without interacting with ransomware infrastructure. *Ransomware.agent* uses DeepSeek as its large language model and operates within a 64,000 token context window constraint. Whilst the model provides strong language understanding and reasoning abilities, it also imposes practical limits due to its context window, response time, cost, and usage quotas. The 64,000 token limit restricts the scale of experiments and data collection, especially when processing content-rich pages with high victim density.

To minimise harm to affected organizations, all victim-identifying information in screenshots has been redacted, including company names, URLs, addresses, and data descriptions. While source material exists on publicly accessible leak sites, this research prioritises responsible disclosure by preserving only UI structure and design elements necessary for analysis.

The project consists of three main stages. **System Design** establishes the architecture for *Ransomware.agent* following a Sense-Plan-Execute model. This includes the high-level architecture and implementation of sensing, planning, and execution modules, alongside the action library that provides primitive browser operations for complex navigation and extraction workflows. **Prompt Engineering** documents the iterative refinement of three critical prompts through qualitative experimentation. This stage showcases how the data extraction, planning, and content summarisation prompts evolved, revealing consistent patterns in which explicit output formatting and domain-specific guidance proved essential for reliable prompts. **Evaluation** measures system performance through two complementary approaches. Coverage measurement confirms the completeness of victim data extraction by comparing against existing victim counts on the sampled DLS, whilst vector similarity analysis uses embedding-based techniques to evaluate how well extracted fields match semantically with ground truth data.

Evaluation across five ransomware groups revealed that extraction viability depends critically on interface design, information density, connection stability, and pagination complexity. For explicitly stated fields, including victim names and domains, the agent consistently achieved 80-100% semantic similarity when information appeared in structured formats. DragonForce reached perfect 100% accuracy, whilst Qilin achieved 83-87% accuracy. Country and industry

fields showed wider variation, achieving 19-89% and 25-46% accuracy respectively. Manual review revealed that lower country scores stem primarily from representational mismatches rather than extraction failure. The agent outputs full country names, whilst conventional parsers extract ISO codes, which causes vector similarity metrics to penalise semantically equivalent but syntactically different outputs. Play's DLS scrape achieved 89% country accuracy when country names appeared explicitly in structured formats, confirming that the agent reliably extracts geographic information when presented clearly.

Coverage results demonstrate critical dependency on site architecture, connection stability, and information density. TheGentlemen achieved perfect 100% coverage across 43 victims. Qilin achieved 95.6% coverage across 457 victims. Play achieved 90.5% coverage across 833 victims. These results validate that the Sense-Plan-Execute architecture matches conventional parser completeness when sites follow standard design principles and network connections remain stable. However, specific architectural patterns and operational conditions create systematic bottlenecks. Akira's 35-page structure resulted in 41.7% coverage due to pagination hallucination across independent site sections. DragonForce achieved 65.0% coverage because 24 victims per page exceeded output token capacity. Pages displaying more than 20-25 victims systematically exceed token limits. Play's 90.5% coverage across approximately 70 pages demonstrates that extensive pagination can be handled successfully. However, the agent remains vulnerable to mid-scrape connection failures and lacks mechanisms to resume interrupted extractions.

These findings confirm that AI-powered extraction represents a viable complementary approach to conventional parsing rather than a complete replacement. The agent excels at rapid deployment against new targets and semantic field extraction from unstructured text. Conventional parsers excel at deterministic navigation and complete coverage through exhaustive iteration. Structured interfaces with clear semantic markup enable reliable extraction with 85-100% field accuracy and 65-100% coverage when connections remain stable. Unconventional interaction patterns like Akira's terminal-style interface degrade coverage substantially.

Future improvements should focus on three areas. First, addressing operational resilience requires implementing error detection mechanisms and stateful execution that enables resumption from interruption points. Play's results demonstrate that connection failures during long-running scrapes represent a critical limitation. Additionally, addressing page density failures requires either deploying models with substantially larger context windows or integrating Retrieval-Augmented Generation to process victim listings in manageable chunks without

exceeding token limits. Second, extending the agent beyond data leak sites to forums and marketplaces requires demonstrating that more abstract prompts can handle diverse site structures. Third, improving evaluation methods requires developing metrics that recognise when extracted values are semantically correct despite differing in format, including accepting "United States" and "US" as equivalent. This also requires establishing clearer standards for what constitutes correct extraction based on actual intelligence analysis needs rather than comparison with existing parsers.

2. Background

Understanding the technical foundations of ransomware operations and modern web scraping techniques is essential to know how *Ransomware.agent* addresses existing limitations in threat intelligence collection. The following subsections cover the key technologies that ransomware groups use to operate anonymously, explain how traditional web scraping works and its limitations, and introduce the language model and agentic workflows that enable agentic web navigation and data extraction. These background topics establish the foundation for the system design and evaluation presented later on.

2.1. Ransomware

A *Ransom* is defined as a transaction exchange demanded for the release of a captive [16]. In the scope of ransomware, the *captive* is the data stolen from the individuals or organisations. Using malware, criminals steal the data and demand a payment to return the data back to the owners. Often in ransomware, the stealing and returning is done through data encryption and decryption, infiltrated using tactics like phishing to install corrupted files for malware to gain access to the data in the victim's computer system [54][17]. Then the data is encrypted by manipulating the data against a key and an encryption algorithm like Rivest Shamir Adleman (RSA) [53], which makes the data unreadable and unusable unless the victim attains the key to decrypt the information back to its original readable state [18]. Hence the perpetrator demands a payment if the victim desires to attain the key to access the breached data.

2.2. Cryptocurrency

The most popular payment option for ransomware is done through Bitcoin. Bitcoin is a decentralised digital cryptocurrency system managed utilising a peer-to-peer (P2P) network of "miners" that validate the transactions occurring in the Bitcoin ledger. This eliminates the requirement of a central authority to manage the transactions between users. Users are represented using a 160-bit hash address which is used as a public key for users to know which address to send the Bitcoin [19]. This creates an anonymous environment where a user can theoretically make new Bitcoin addresses and send over the public key address to receive payments without using any form of public identity.

2.3. TOR Browser

TOR is an anonymity network that relays traffic through a volunteer operated circuit so that no single hop observes both the origin and the destination of a connection [20]. Each relay decrypts only its own layer of the payload before forwarding the remainder, preventing intermediate nodes from reading the plaintext or learning the user's identity [20]. TOR further encapsulates

traffic into uniform 512 byte "cells," a defence against traffic analysis techniques that rely on packet length fingerprints [21]. Beyond anonymising clients, the protocol also supports hidden services that advertise .onion addresses so servers can remain anonymous while users connect without revealing their IP addresses [20]. Ransomware actors embraced TOR to shield command infrastructure and payment portals from law enforcement intervention. CTB-Locker, observed in 2014, routed victims to TOR based hidden services for both key delivery and ransom payment, hindering takedown efforts [23]. By 2019 the Maze collective introduced double extortion, threatening to leak stolen files unless payments were made, and subsequent groups such as Sodinokibi/REvil, DoppelPaymer, Nefilim, and Clop established leak sites on TOR to amplify pressure on targets [24]. These .onion portals inherit TOR's anonymity and resilience, making operators hard to identify and their servers hard to seize or block [22].

2.4. Ransomware as a service (RaaS)

Ransomware-as-a-Service (RaaS) packages extortion tooling, payment infrastructure, and victim negotiation playbooks into a subscription offering that lowers the barrier of entry for aspiring criminals. Core developers operate affiliate platforms, sell access to builder kits, and take a percentage of successful payouts while partners contribute intrusion skills or compromised network access [10]. Franchising the tooling in this way dramatically increases the number of operators targeting enterprises, which overwhelms manual threat monitoring teams and accelerates the rate at which new leak sites appear [25]. An adaptive collection pipeline must therefore cope with a rapidly expanding and heterogeneous set of HTML structures.

2.5. Data Leak Sites (DLS)

Leak sites host the "name-and-shame" component of modern double extortion campaigns. Groups such as LockBit [55] and Akira [56] post stolen documents, countdown timers, and negotiation transcripts in highly customised layouts that are regularly tweaked to disrupt rival scrapers and keep media attention focused on new victims [14]. Aggregators have documented the steady growth of these sites, noting that 2024 saw record numbers of launches and redesigns that reshaped page structure and imposed new anti-automation obstacles [26]. Collecting reliable intelligence therefore requires a parser that understands both the semantics and the visual hierarchy of each page without relying on brittle CSS selectors.

2.6. Web Scraping and Parsing HTML

Early web scraping techniques were largely rule based and relied on textual pattern matching. In the 1990s, developers often wrote site specific wrappers that treated a webpage as plain text and searched for data using regular expressions [27]. This approach was straightforward but labour intensive because the rules had to be tailored to each site's HTML markup and would break

easily if the page structure changed. As websites grew more complex, scraping methods evolved to leverage the HTML structure itself. Rather than using only text patterns, tools emerged to parse pages into a structured format called the Document Object Model (DOM). This shift enabled scrapers to navigate content by HTML tags and hierarchy instead of raw text. The mid 2000s saw the introduction of libraries like BeautifulSoup [57] that greatly simplified HTML parsing by automatically building a DOM tree from a page and letting programmers traverse it to find elements [28]. These developments marked a transition from ad hoc scripts to more robust DOM based scraping techniques.

Modern traditional web scrapers build on these foundations by using the page's DOM along with query languages or selectors to extract information. Developers can traverse the DOM tree using XPath queries or CSS selectors to pinpoint specific elements of interest [29]. For example, a script might fetch a webpage, parse it into a DOM with a library such as BeautifulSoup, and then retrieve data by selecting elements by tag name, CSS class, or position in the HTML hierarchy. All these methods inherently depend on the webpage's underlying HTML layout, which means scrapers can be fragile. Even minor changes in a site's markup may cause an extraction rule to fail because the selectors no longer find the expected content. Ferrara et al. (2014) highlight that a small structural modification in a page often causes a previously working scraper to stop functioning [15]. This fragility remains a known drawback of traditional site-specific scrapers, which requires developers to continually maintain and update scraping rules as websites evolve.

2.7. Large Language Models

Large Language Models (LLMs) offer an alternative path by reasoning over raw text and HTML tokens, allowing a parser to generalise from prompts rather than static selectors. Recent studies show that foundational models can infer document structure, label elements, and describe rendered content with high accuracy, even without fine-tuning, which makes them suitable for interpreting leak site layouts that evolve weekly [30]. Harnessing these capabilities requires carefully engineered prompts and safeguards that transform unstructured HTML into consistent JSON for downstream analysis.

2.7.1. LLMs use for HTML

A study done by the Google research team, Gur et el. (2023) discussed the implications of a pretrained LLM that was unchanged and focused on its ability to interpret raw HTML considered as a sequence of tokens. Utilising three different WebD models, an evaluation on "description generation" was conducted to test the model's ability to understand the HTML structure as it would appear to the user, despite not having access to the web interface. The

results showed an average score of around 84% in accuracy, where other fine-tuned models showed accuracies up to 91% [30]. Although this proposal does not look to implement fine-tuning models, this research shows the potential to utilise LLMs to interpret absolute raw HTML to automate parsing generation and reducing manual maintenance

2.7.2. Prompt Engineering

A study by Clairoux-Trépanier et al. (2024) conducted a thorough review of conversations interpreted by LLMs and evaluated on its accuracy. The system's core functionality relied on the use of carefully curated prompts that assist the LLM to extract the specific information. This example prompt was used to show the intent and requirements for the data extraction.

```
"As a cyber threat intelligence analyst, your task is to review the
    conversation and identify key indicators. Please extract the following
    information: 1. An actor is selling something? 2. The conversation involves
     the sale of initial access to a corporate or organization network? 3.
    Targeted or abused products or technology names?". Output your answer in
    the following format, as an example.

{
 "summary": "The conversation focused on the sale of a new exploit targeting
     XYZ software. An actor claimed to have discovered a vulnerability and is
     offering it for sale. There was also a discussion about potential targets
     using ABC technology.",
 "variables": {
     "is_sale": true,
     "is_initial_access": true,
     "targeted_technologies": "XYZ software, ABC technology"
 }
```

Figure 1: Example prompt for the LLM [31]

Using the GPT-3.5-turbo model it performed at an average accuracy score of 96.23%, an average precision of 90% and an average recall of 88.2%. Limitations addressed improvements that should have been made in the prompts, which include the consideration of verb tenses and minimising the use of large vague concepts [31]. In the case of this proposal a focus on designing a prompt that allows for modularity and the ability to interpret specific details is key to having an adaptable LLM-powered parser. The results in Clairoux-Trépanier et al. (2024) show that it is possible using "is_sale" related semantic reasoning, which can extend to other use cases.

2.8. Agentic Workflows

Agentic workflows describe AI systems that can work independently by repeatedly observing their environment, deciding what to do next, and taking action [32]. This approach is borrowed from robotics, where a robot follows a Sense-Plan-Act loop [58][59]. First it observes its surroundings, then it reasons about what should happen next, and finally it performs an action. After each action, the system checks what changed and adjusts its plan accordingly.

This pattern is particularly useful for web navigation because it allows an AI agent to adapt to unexpected situations. Unlike traditional automation, which follows a fixed set of instructions and breaks when something changes, agentic systems can adjust their behaviour when they encounter new page layouts or unexpected content [32][33]. Recent research shows that large language models can power these agents by breaking down complex goals into smaller steps and reasoning about what to do at each stage. For ransomware leak site monitoring, this means *Ransomware.agent* can navigate unfamiliar website structures, decide when to extract data, and handle changes to page layouts without requiring manual updates to the code. The agent continuously learns from what it sees and adjusts its strategy accordingly.

3. Literature Review

Existing research in ransomware threat intelligence and autonomous web navigation establishes the context for this work. Security analysts currently rely on manual parsers to monitor ransomware leak sites, while recent developments in AI-powered web navigation systems have demonstrated the viability of agentic approaches. Understanding current monitoring tools shows where manual maintenance becomes a problem, while examining agentic architectures reveals design patterns that *Ransomware.agent* can adopt.

3.1. Ransomware DLS monitoring

Several community-run platforms currently monitor ransomware leak sites, including Ransomware.live, RansomWatch, and RansomLook [11][12][13]. These projects work by regularly visiting leak sites through the TOR network, saving copies of the web pages, and using custom parsers to extract victim information. Each parser is hand-written to match a specific ransomware group's website structure. While this approach works well when sites stay the same, it creates a maintenance burden because parsers break whenever a group redesigns their site.

Figure 2 shows how Ransomware.live organizes its code. The system separates the task of downloading web pages from the task of parsing them. One component handles connecting to TOR and saving HTML files, while a separate directory contains individual parser scripts for each ransomware group. These parsers use a Python library called Beautiful Soup to navigate HTML and extract victim data. This design makes the code easier to understand, but each parser still needs manual updates when websites change [15].

```
Ransomeware.live
|-  libs/
|    |-  ransomwarelive.py
|    |-  ..
|    |-  ..
|-  parsers/
|    |-  ..
|    |-  akira.py
|    |-  ..
|    |-  clop.py
|    |-  ..
```

Figure 2: Ransomware.live file directory [13]

The Akira ransomware group provides a clear example of this challenge. In mid-2025, Akira used a website that looked like a command-line terminal interface. The HTML structure was relatively simple, using repetitive div tags and pre-formatted JSON blocks to display victim information. Figure 3 shows what this looked like. Ransomware.live built a custom parser called akira.py as seen in Algorithm 1 that searched for these pre tags, extracted the JSON content, and saved victim records to a database. The parser met the requirements as it was designed specifically for that HTML structure.

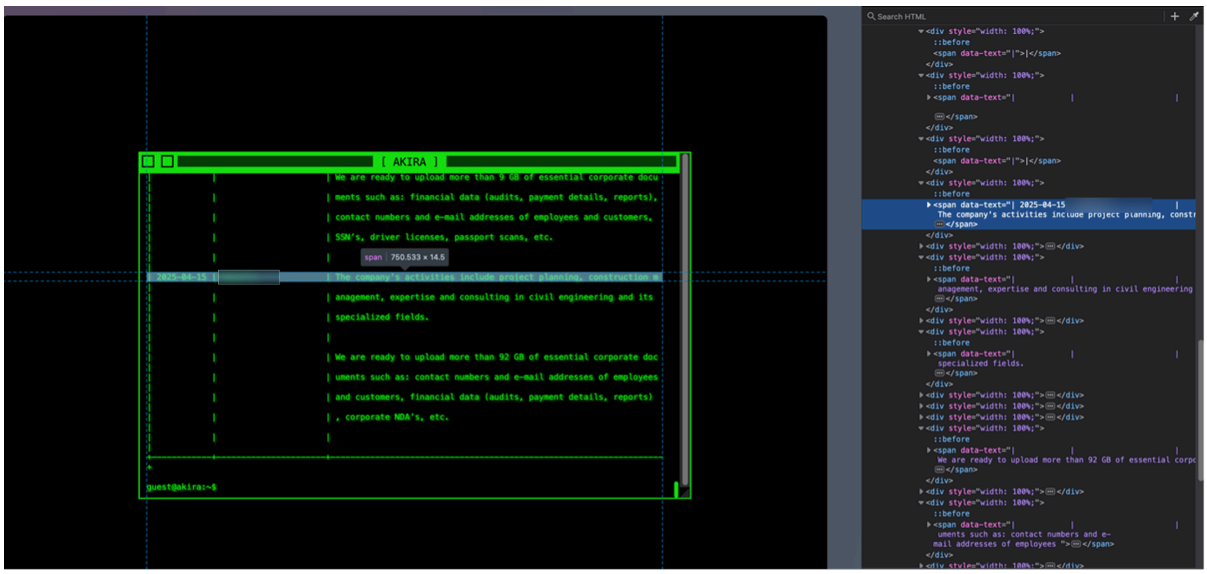


Figure 3: Akira ransomware website and HTML snippet in April 2025

Algorithm 1: Akira HTML Parser

```
1:  procedure main(source_dir)
2:      for "filename in source directory" do
3:          if "filename starts with "akira-"" then
```

```
        “html_doc” ← “open file at path “source/” + filename”
        “soup” ← “parse html_doc with BeautifulSoup”
        “jsonpart” ← “extract contents from tag”
        “data” ← “parse jsonpart as JSON”
        for “entry in data” do
            “title” ← “unescape HTML in entry.title”
            “date_str” ← “entry.date”
            “description” ← “entry.content”
            “dt” ← “parse date_str as date (YYYY-MM-DD)”
            “now” ← “current time”
            “published” ← “combine dt’s date with now’s time”
            “published_str” ← “format published as string”
            appender(title, “akira”, description, “”, published_str)
        end
    end
end
```

In October 2025, Akira completely redesigned their site. The new version used a paginated layout with content loaded through an API, which made the old parser completely useless. Figure 4 shows the updated design.

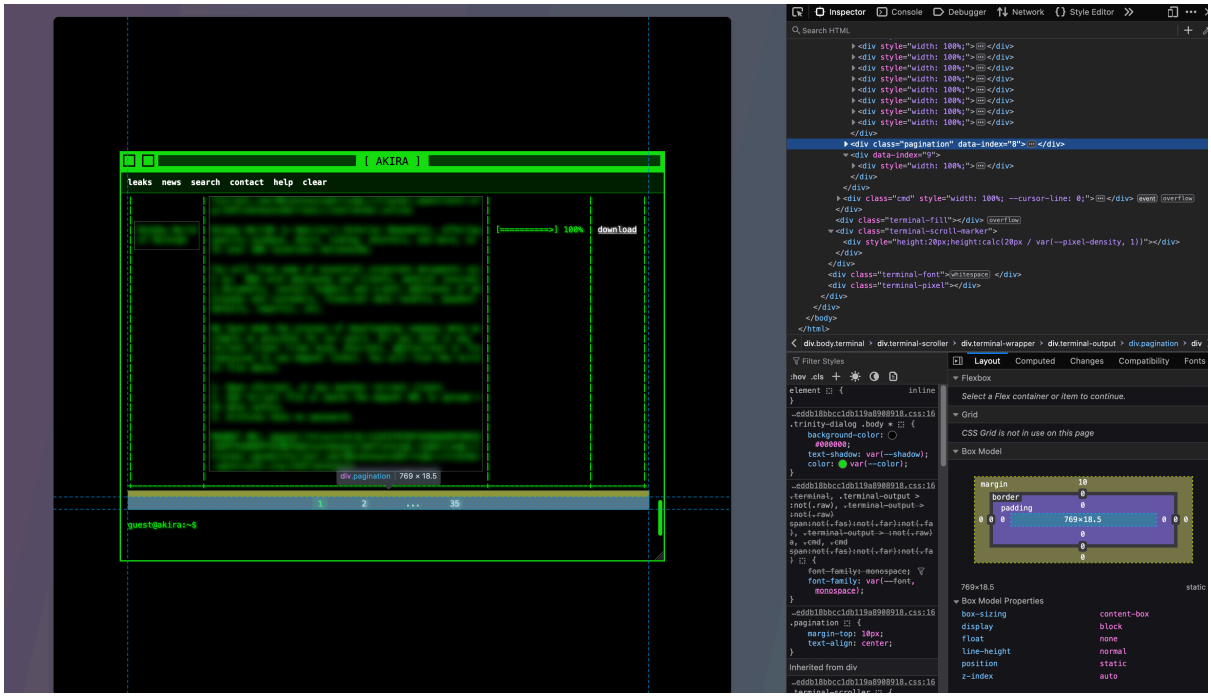


Figure 4: Akira ransomware website and HTML snippet in October 2025

This enforced a change in Ransomware.live's parser to a new API driven approach summarised in Algorithm 2. While this new parser is effective, this situation illustrates the maintenance problem described by Ferrara et al. (2014). Even small changes to a website can break parsers that rely on specific HTML structures [15].

Algorithm 2: Akira API Parser After New DLS Design

```
1:  procedure main()
2:      "news_url" ← "Akira news API endpoint"
3:      "leak_url" ← "Akira leak API endpoint"
4:      "site_url" ← "Akira onion homepage"
5:      "auth" ← "Get-CSRF-Token(site_url)"
6:      "csrf_token" ← "auth.token"
7:      "cookies" ← "auth.cookies"
8:      if csrf_token != null and cookies != null then
9:          "headers["X-CSRF-Token"]" ← "csrf_token"
10:         for "url in (news_url, leak_url)" do
11:             "data" ← "Fetch-JSON(url, cookies)"
12:             if data != null then
13:                 for "entry in data["objects"]" do
14:                     if url == news_url then
15:                         "timestamp" ← "entry.date + " 00:00:00.000000"""
16:                         appender(entry.title, "akira", entry.content, "", timestamp, "")
17:                         appender(entry.name, "akira", entry.desc)
18:                     end
19:                 end
20:                 errlog(Failed to parse API response)
21:             end
22:         end
23:         errlog(Unable to obtain CSRF token or cookies)
24:     end
25: end
```

These community platforms do impressive work, but they all share the same fundamental problem, which is their dependence on volunteers who must constantly update parsers whenever websites change. This Akira redesign shows that the current approach relies on people quickly noticing changes and writing new code, rather than systems that can adapt automatically. This creates a clear gap in the research, as monitoring tools need to be able to understand the meaning of web content rather than just matching HTML patterns. Using language models and agentic workflows could reduce the maintenance burden while still keeping the security and ethical safeguards that these projects already follow.

3.2. Agentic Web Navigation: Agent-E

Agent-E is a recent system that demonstrates how AI agents can navigate websites autonomously. Figure 5 shows the basic concept behind web agents. They need the ability

to sense what is on a web page and the ability to act by clicking buttons, filling forms, and navigating links.

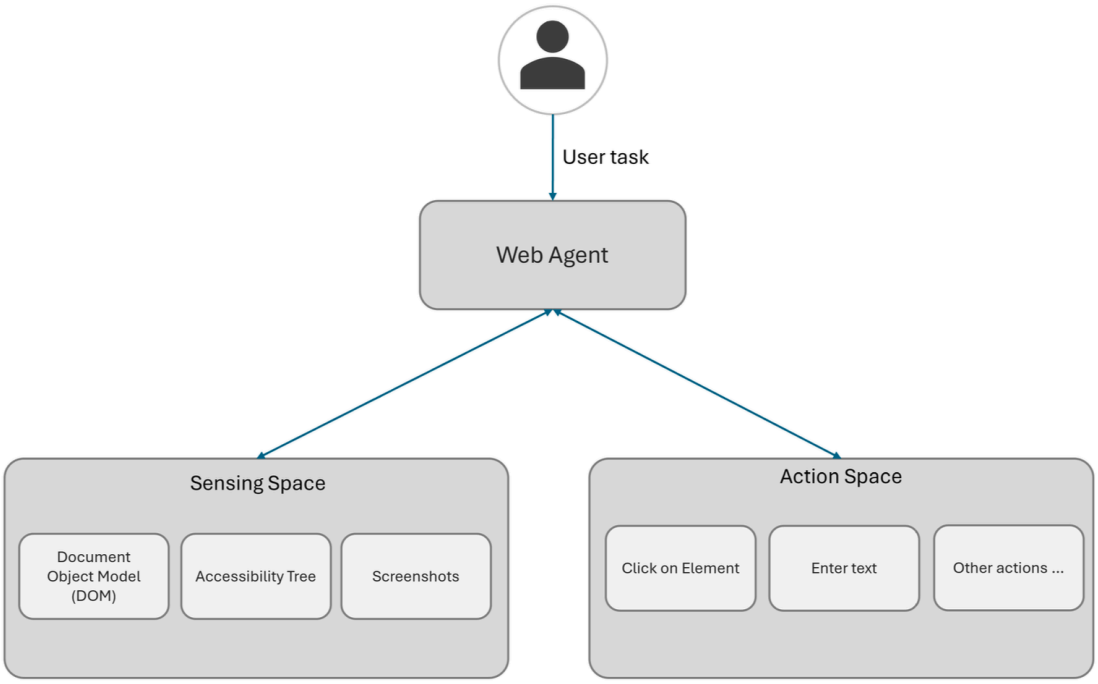


Figure 5: Fundamental Diagram of an Agentic Web Navigation Framework [34]

Sensing means capturing what is currently on the page by reading the HTML structure, checking the accessibility tree, or taking screenshots. Acting means performing operations like clicking links, typing into forms, or moving between pages. Agent-E builds on this concept using two coordinated components powered by large language models (a Planner agent and a Browser Navigation agent). The Planner interprets the user's goal and breaks it down into smaller steps. The Browser Navigation agent receives these steps one at a time and executes them using web interaction skills. Figure 6 shows how this works as a loop. The Planner decides what to do, the Browser agent performs the action, and then reports back what happened. This design keeps the Planner focused on high-level strategy while the Browser agent handles the details of page interaction.

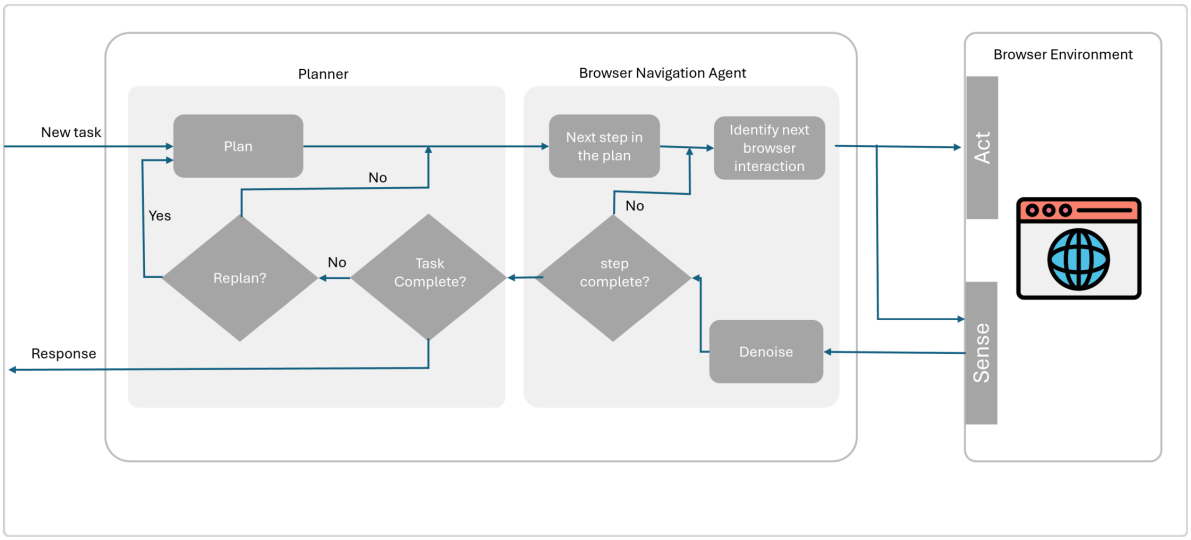


Figure 6: Agent-E conceptual flow diagram [34]

Agent-E uses Playwright, a browser automation library, to control web pages programmatically [35]. The Browser Navigation agent wraps Playwright operations in Python functions that act as "skills" the agent can use. The language model decides which skill to use based on what it sees on the page and what task it needs to complete [34]. The table below lists the basic browser skills that Agent-E provides. This design pattern influenced how *Ransomware.agent* is structured, with a similar separation between planning logic and browser interaction functions.

Skills	**Input parameter**	**Change observation during skill execution**	**Return**
Get DOM	content type: text_only	None	Returns the inner text of the HTML body element with limited post-processing. Ideal for text summarisation and information extraction.
	Content type: input fields	None	Returns a JSON representation of specific HTML elements such as buttons, input fields, and links. Ideal for interacting with search fields or buttons.
	Content type: all fields	None	Returns a JSON representing the full page. Most complete representation of all elements, but also the most lengthy and noisy.
Click	Selector: identifier of the element to be clicked.	Observe for DOM change events immediately following the click.	Returns a textual response indicating whether the click was performed and summarising any changes observed (if any).
Enter text	Selector: identifier of the element that receives text.	Observe for DOM change events during or following the text input.	Returns a textual response indicating whether text input was performed and summarising any changes observed (if any).
Open URL	URL: the address to navigate.	Web navigation.	Returns the page URL and title of the new page.
Press key	Keys to press (for example, Submit, PageDown).	Observe for DOM change events immediately following the key press.	Returns a textual response indicating whether the key press was performed and summarising any changes observed (if any).

Table 2: Agent-E browser navigation skill library [34]

4. Methodology

The methodology is organised into three parts. System Design explains the architecture and core skills that power the agent, including the high-level architecture, sensing, planning, and execution modules, as well as the action library that enables browser interactions. Prompt Engineering documents the evolution of critical prompts through systematic testing against diverse ransomware leak sites, covering data extraction, strategic planning, and DOM summarisation techniques. Evaluation describes the methods used to measure extraction accuracy and completeness against established ransomware data leak sites.

4.1. System Design

Following the Sense-Plan-Action paradigm introduced in the agentic workflow literature, the agent repeatedly observes its environment, reasons about what to do next, and performs actions until it achieves its goal. This section first presents the high-level architecture that coordinates these phases, then explores each of the core modules: sensing, planning, and exectue that enable the agent to operate adaptively. Finally, it describes how these skills are integrated into a working execution pipeline.

4.1.1. High Level Architecture

Ransomware.agent is built as a modular pipeline where each stage handles a distinct responsibility. The architecture separates concerns so that the sensing module focuses on understanding the current web page, the planning module decides which action to take next, and the execution module executes browser operations. These modules are powered by DeepSeek's proprietary large language model DeepSeek-R1, accessed through the DeepSeek SDK [37], which interprets HTML content and generates reasoning for navigation decisions. DeepSeek-R1 operates with a 64,000 token context window for both input and output [37]. This context limitation becomes particularly critical when processing ransomware leak sites that display high victim density per page, as the model must fit both the input HTML content and the structured output JSON within this 64,000 token budget.

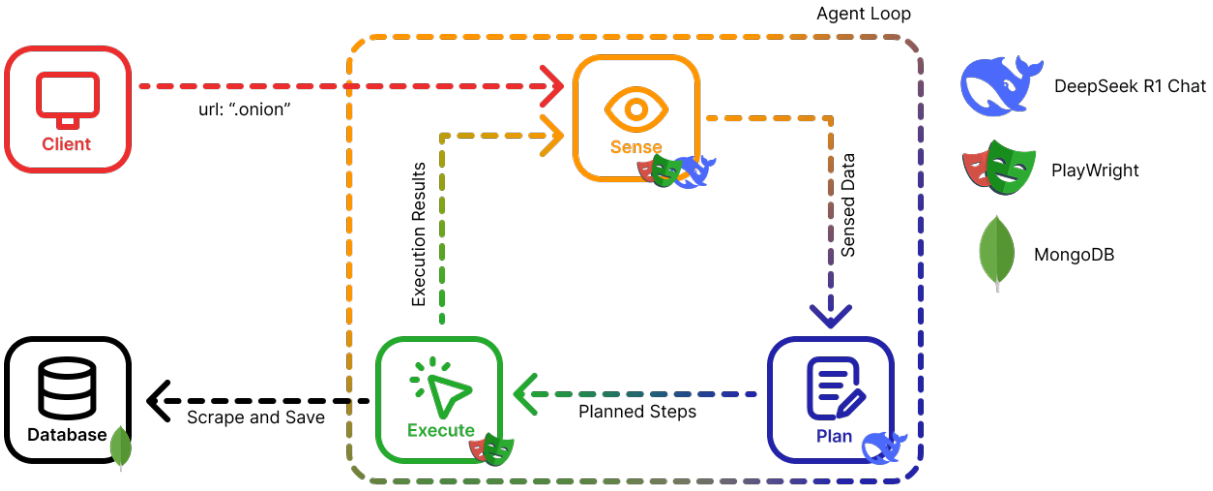


Figure 7: *Ransomware.agent* high-level architecture

Figure 7 illustrates this modular architecture, showing how the system orchestrates the Sense-Plan-Execute cycle through distinct components. The Sensing module uses Playwright to read the raw Document Object Model (DOM) and capture the full HTML content of the current page. It then invokes the language model to summarise the key elements within the DOM, compressing context and muting noise to produce actionable information for analysis. The Planning module receives this processed information, consults the action history, and invokes DeepSeek-R1 to reason about which action to perform next. The Execution module then dispatches the chosen action to the appropriate handler, which interacts with the browser through Playwright. The system uses Playwright, a cross-platform browser automation library, as its primary execution tool for performing browser operations [35]. Playwright provides programmatic control over Chromium-based browsers, enabling the agent to click elements, extract text, navigate pages, and take screenshots without manual intervention. This automation layer translates the agent's high-level action decisions into concrete browser interactions.

Extracted victim data and leaked intelligence are stored in MongoDB, a NoSQL database that natively supports JSON document storage [36]. This design decision aligns with the data structure used by Ransomware.live, which also stores victim records as JSON documents. The structural compatibility enables direct comparison between the agent's outputs and the ground truth data collected by Ransomware.live's parsers during evaluation. MongoDB's schema-less document model accommodates the varying field structures across different ransomware groups without requiring predefined table schemas, which suits the heterogeneous nature of leak site data where different groups present victim information in different formats.

Algorithm 3 outlines the control flow of the agent's main execution loop. The agent starts by connecting to a ransomware leak site through a headless browser. It then enters a loop where it

senses the current page, builds a planning context, asks the model to propose the next action, executes that action, and updates its history. This cycle continues until the agent decides it has extracted all available data or reaches a maximum step limit.

Algorithm 3: #emph[Ransomware.agent] Main Control Loop

```
procedure run_agent(start_url, model, job_id, max_steps)
    ▷ Initialize managers and browser
    "llm" ← "LLMManager(model)"
    "browser" ← "BrowserManager(start_url, job_id, llm)"
    browser.start()
    "planning_manager" ← "PlanningManager()"
    "steps" ← "0"
    "exhausted_navigation" ← "false"
    while "steps < max_steps" do
        "steps" ← "steps + 1"
        ▷ SENSE: Capture and process current page state
        browser.sense()
        "page" ← "browser.get_page()"
        "full_text" ← "page.content()"
        "url" ← "page.url"
        "text_hash" ← "SHA256(full_text)"
        "seen_before" ← "check_session_hash(url, text_hash)"
        "sensing_context" ← "browser.sensingcontext"
        ▷ PLAN: Build context and generate action plan
        "context" ← "planning_manager.build_context(browser, seen_before)"
        "plan" ← "planning_manager.plan(context, llm)"
        "actions" ← "parse_plan_to_actions(plan)"
        "stop_requested" ← "planning_manager.register_plan(actions)"
        if stop_requested or actions is empty then
            "exhausted_navigation" ← "true"
            break
        end
        ▷ EXECUTE: Perform actions and update history
        "results" ← "browser.execute(actions)"
        "mapped_results" ← "map_actions_to_results(actions, results)"
        planning_manager.update_history(steps, mapped_results)
    end
    browser.exit()
end
```

4.1.2. Sense

Capturing and processing the current state of each web page is critical because raw HTML must be transformed into actionable information that the planning module can reason about. At every iteration of the agent's control loop, the system must understand both the content and structure of what it observes.

At the start of each cycle, the agent captures the page's full HTML content using Playwright's page accessor methods. The raw HTML typically contains many elements that are not useful

for navigation or extraction, such as inline scripts, style declarations, advertisements, and tracking pixels. The sensing module applies a cleaning process that removes these elements while preserving the semantic structure and textual content.

Algorithm 4 shows the pseudo code for the core sensing logic. The algorithm retrieves the page content, applies HTML cleaning to remove noise, and checks whether the cleaned content fits within the model's 64,000 token context window. If the content is too long, it applies a chunking strategy with overlap to ensure no information is lost at chunk boundaries.

Algorithm 4: Sensing Phase

```
procedure sense(page)
    "raw_html" ← "page.content()"
    "cleaned_html" ← "clean_html(raw_html)"
    "text_content" ← "extract_text(cleaned_html)"
    "url" ← "page.url"
    "title" ← "page.title()"
    if length(text_content) > MAX_CONTEXT_WINDOW then
        "chunks" ← "chunk_text(text_content, overlap=0.1)"
        "summaries" ← "[]"
        for "chunk in chunks" do
            "summary" ← "llm.summarize(chunk)"
            summaries.append(summary)
        end
        "final_summary" ← "merge_summaries(summaries)"
        "context" ← "final_summary"
        "context" ← "text_content"
    end
    "interactive_elements" ← "extract_interactive(cleaned_html)"
    "content_hash" ← "SHA256(text_content)"
    ▷ Create SensingContext with all fields
    "sensing_context" ← "SensingContext()"
    "sensing_context.context" ← "context"
    "sensing_context.url" ← "url"
    "sensing_context.title" ← "title"
    "sensing_context.interactive_elements" ← "interactive_elements"
    "sensing_context.content_hash" ← "content_hash"
    return sensing_context
end
```

The output is a SensingContext object that includes the processed page content, the current URL and page title, a list of interactive HTML elements, and a content hash for page deduplication. This structured representation provides everything the planning module needs to make informed decisions. When content exceeds the model's 64,000 token context window, the chunking strategy divides text into overlapping segments. Each segment is independently summarised by the model, and the summaries are then merged into a coherent summary again. This approach

maintains coverage of long victim lists without losing critical details that might appear at chunk boundaries.

4.1.3. Plan

Deciding what action to take next requires synthesising the current sensing context with the agent's execution history to select an appropriate response. Planning is implemented as a language model query where the prompt provides the agent's objective, current observations, available actions, and past results. Algorithm 5 outlines the planning logic. The planner builds a comprehensive prompt that includes the sensing context, action history, and detailed descriptions of each available action. The language model is instructed to return a single JSON object specifying which action to take and why.

Algorithm 5: Planning Phase

```
procedure plan(sensing_context, history, llm)
    "seen_before" ← "check_visited(sensing_context.content_hash)"
    if seen_before then
        "stop_action" ← "Action(type="stop")"
        return stop_action
    end
    ▷ Build planning prompt with all context
    "prompt" ← "PlanningPrompt()"
    "prompt.objective" ← ""Extract all victim records""
    "prompt.current_page" ← "sensing_context.url"
    "prompt.page_content" ← "sensing_context.context"
    "prompt.interactive_elements" ← "sensing_context.interactive_elements"
    "prompt.action_library" ← "get_action_descriptions()"
    "prompt.history" ← "history"
    "response" ← "llm.query(prompt, format="json")"
    "action" ← "parse_json(response)"
    validate_action(action)
    register_visit(sensing_context.content_hash)
    return action
end
```

The planning prompt is carefully structured to guide the language model toward making strategic decisions. The prompt provides context about the current page state, available actions from the action library (detailed in Section 4.1.5), and execution history, enabling the model to reason about which action to take next. The structure and iterative refinement of this planning prompt are discussed in detail in Section 4.2.2, where explicit behavioural constraints and termination logic proved essential for reliable navigation. The model analyses this information and returns a JSON response with three fields, which are action type, parameters, and reasoning. When the agent identifies a page containing victim records, the response might look like this.

```
{
  "action": "scrape_and_store",
  "parameters": {},
  "reason": "Page contains victim list with company names and descriptions"
}
```

The planner includes deduplication logic that tracks content hashes of previously visited pages. If the same content is encountered again, the planner returns a "stop" action to prevent infinite loops. This is essential for handling sites with circular navigation or dynamic content that doesn't change between visits.

4.1.4. Execute

Translating planning decisions into actual browser operations requires a robust execution framework that manages dynamic action dispatch, error recovery, and result aggregation. The execution module in the BrowserManager class receives a list of planned actions and orchestrates their sequential execution, handling both standard browser interactions and specialized operations like data scraping. Algorithm 6 shows the execute method that dispatches actions to their corresponding handlers.

Algorithm 6: Action Execution Dispatcher

```
1:  procedure execute(actions)
2:      "results" ← "[]"
3:      for "action in actions" do
4:          "name" ← "action.get("action")"
5:          "selector" ← "action.get("selector")"
6:          "params" ← "action.get("params", {})"
7:          "mod" ← "import_module(actions[name])"
8:          if "name == "scrape_and_store"" then
9:              "result" ← "await mod.run(page, victims_collection, session_collection, llm)"
10:             "result" ← "await mod.run(page, selector, params)"
11:         end
12:         "formatted" ← "format_action_outcome(name, result)"
13:         results.append(formatted)
14:     end
15:     return results
16: end
```

The execute method implements a dynamic dispatch pattern using Python's importlib to load action modules at runtime. Each action type (click, enter_text, scroll_to, etc.) corresponds to a separate Python module that encapsulates the Playwright operations needed to perform that action. This design allows for easy extension of the action library without modifying the core execution logic.

The method handles two distinct execution paths: standard actions that accept a selector and parameters, and the specialized scrape_and_store action that requires access to database collections and the LLM instance. For standard actions, arguments are prepared based on whether a selector is provided and the specific requirements of the action type. The wait action, for example, treats its selector as a duration parameter rather than a DOM selector.

Error handling is critical for robust execution in the unpredictable environment of ransomware DLS sites. Each action execution is wrapped in a try-except block that captures any exceptions, logs them for debugging, and returns a failed status while allowing subsequent actions to proceed. This ensures that a single failed action doesn't halt the entire navigation session.

After each action completes, its outcome is formatted into a standardized result object containing a status field (success or failed), a human-readable message describing what occurred, and an optional payload with extracted data or state information. These results are collected and returned as a list, providing the agent with feedback about which operations succeeded and enabling intelligent recovery strategies in future planning cycles.

4.1.5. Action Library

Equipping the agent with a set of primitive browser operations allows the planning module to combine these skills and achieve complex navigation and extraction goals. Each action is designed to be atomic and composable, enabling the planner to orchestrate sophisticated navigation and data extraction workflows. Table 3 lists all available actions with their parameters and expected outcomes.

Action	Parameters	Operation	Return
click	selector: CSS selector or XPath	Locates the element and simulates a click. Waits for navigation or DOM updates.	Success status, page changes, new URL if navigation occurred
enter_text	selector: element selector text: string to input	Finds an input field and types the specified text. Simulates keyboard events.	Success status, field value confirmation
scroll_to	selector: element selector OR direction: “up”/“down”	Scrolls viewport to reveal the target element or scrolls by a fixed amount.	Success status, new visible content
wait	duration: milliseconds	Pauses execution to allow dynamic content to load or animations to complete.	Success status, elapsed time
press_key	key: key name (Enter, PageDown, Tab, etc.)	Simulates pressing a keyboard key. Useful for form submission and pagination.	Success status, page changes
go_back	None	Navigates to the previous page in browser history.	Success status, previous URL
handle_dialog	action: “accept” or “dismiss”	Responds to JavaScript alert, confirm, or prompt dialogs.	Success status, dialog text
take_screenshot	path: file path	Captures a screenshot of the current viewport for debugging.	Success status, file path
scrape_and_store	None	Extracts victim records from current page and saves to database. See Algorithm 7 for details.	Success status, number of records extracted, extraction errors
stop	None	Signals that the agent should terminate the execution loop.	Success status

Table 3: *Ransomware.agent* action library

Each action is implemented as a Python function that takes the browser instance and action parameters as input. The functions are organised in a directory, with one module per action type. This modular design makes it easy to add new actions or modify existing ones without affecting the core Sense-Plan-Execute loop. Actions report their results using a standardised Result object that includes a success flag, descriptive message, any extracted data, and error information. This consistent interface allows the planning module to interpret outcomes uniformly. Among all available operations, the scrape_and_store action stands out as the most sophisticated because it orchestrates the full pipeline of content extraction, parsing, validation, and database persistence, with its effectiveness depending critically on prompt engineering techniques.

Algorithm 7 shows the detailed logic for scrape_and_store. The process involves multiple stages, including content capture, chunking if necessary, model-based extraction, JSON parsing with error recovery, deduplication, and database insertion.

Algorithm 7: Scrape and Store Action

```
procedure scrape_and_store(page, llm)
    "url" ← "page.url"
    "html" ← "page.content()"
    "text" ← "extract_text(html)"
    "text_hash" ← "SHA256(text)"
    if is_already_scraped(url, text_hash) then
        "result" ← "Result(success=true, count=0, message="Already scraped")"
        return result
    end
    "chunks" ← "chunk_text(text, overlap=0.1)"
    "all_victims" ← "[]"
    for "chunk in chunks" do
        "prompt" ← "build_extraction_prompt(chunk)"
        "response" ← "llm.query(prompt)"
        "victims" ← "parse_json_with_repair(response)"
        for "victim in victims" do
            validate_victim_fields(victim)
            all_victims.append(victim)
        end
    end
    "unique_victims" ← "deduplicate_by_name(all_victims)"
    "saved_count" ← "0"
    for "victim in unique_victims" do
        ▷ Create database record for victim
        "record" ← "VictimRecord()"
        "record.url" ← "url"
        "record.name" ← "victim.name"
        "record.website" ← "victim.website"
        "record.description" ← "victim.description"
        "record.published" ← "victim.published"
        "record.country" ← "victim.country"
        "record.industry" ← "victim.industry"
        "record.scraped_at" ← "current_timestamp()"
        if not exists_in_db(record) then
            database.insert("victims", record)
            "saved_count" ← "saved_count + 1"
        end
    end
    mark_as_scraped(url, text_hash)
    ▷ Create result summary
    "result" ← "Result()"
    "result.success" ← "true"
    "result.count" ← "saved_count"
    "result.total_extracted" ← "length(all_victims)"
    "result.message" ← ""Extracted and saved victims""
    return result
end
```

The extraction prompt instructs the model to identify victim records in the page content and format them as JSON. The model occasionally produces malformed JSON, so the implementation includes a repair mechanism that attempts to fix common syntax errors before parsing.

Deduplication occurs at two levels by comparing URL and content hash against a scraping log, and by deduplicating victims by name after extraction. Each victim record is timestamped with the scraping time and associated with the source URL.

4.2. Prompt Engineering

The effectiveness of *Ransomware.agent* depended critically on how instructions were communicated to the language model. Early prompts were often too general, leading the model to propose actions that were technically valid but strategically poor. Through iterative experimentation with diverse ransomware leak site structures, three key prompts underwent significant refinement: the data extraction prompt in scrape_and_store, the strategic planning prompt in planningContext, and the DOM summarisation prompts in browser_manager. This section documents each evolution with concrete examples and explains how specific changes improved reasoning quality. A consistent pattern emerged across all prompt refinements, in which explicit output format constraints and domain-specific behavioural guidance proved essential for reliable agent performance and produced emergent reasoning capabilities such as logical action chaining, improved error recovery, and robust semantic understanding across diverse HTML structures. The refinements described here were developed through manual qualitative testing, which involved observing agent behaviour on diverse ransomware DLS.

4.2.1. Scrape and Store

The initial design for the scrape_and_store action prompt focused on comprehensive field extraction but lacked specificity about output formatting requirements, leading to inconsistent JSON responses that required extensive error handling and manual correction. The original prompt requested a broad set of fields without clear guidance on field naming conventions, as shown in Listing 1.

```
prompt = (f"""
  Given the current chunk of text and its surrounding context,
  extract a list of valid entries. Return a JSON list; each entry should have:
  'post_title', 'ransomware_group_name', 'discovered_timedate', 'description',
  'industry', 'published_timedate', 'post_url', 'country', 'ransomware_activity',
  'company_website', 'duplicates'.\n\n

  PREVIOUS CHUNK:\n...\n\n
  CURRENT CHUNK:\n...\n\n
  NEXT CHUNK:\n...\n\n
  --- OMISSIONS ---
    - Do NOT include any explanation, commentary, or formatting outside of the JSON list.
""")
```

Listing 1: Original extraction prompt with eleven fields and no output format constraints

This prompt produced several problems. The model frequently returned JSON wrapped in markdown code blocks (enclosed in ```json fences), which broke the JSON parser and required

additional text preprocessing to strip formatting markers. The field names were verbose and inconsistent (for example, “ransomware_group_name” versus “company_website”), which confused the model’s understanding of context and led to inconsistent outputs that were poorly correlated with the intended field meanings. Additionally, the initial schema included fields such as “duplicates” and “ransomware_activity” inherited from the Ransomware.live data structure, but these fields were removed as they fell outside the scope and relevance of this research.

The refined prompt addressed these issues through three strategic changes, which were simplified field schema, semantically clearer field names, and explicit output format constraints, as shown in Listing 2.

```
prompt = (f"""
  Given the current chunk of text and its surrounding context,
  extract a list of valid entries. Return a JSON list; each entry should have:
  'industry', 'attackDate', 'countryOfCompany', 'description', 'discovered',
  'companyWebDomain', 'ransomwareGroup', 'victimCompany'

  PREVIOUS CHUNK:\n...\n\n
  CURRENT CHUNK:\n...\n\n
  NEXT CHUNK:\n...\n\n
  --- OMISSIONS ---
    - Do NOT include any explanation, commentary, or formatting outside of the JSON list.
    - Return the output without any markdown formatting (no code fences, white space).
""")
```

Listing 2: Refined extraction prompt with eight semantically clear fields and explicit formatting constraints

Field names were redesigned to be semantically clearer, directly conveying their intended meaning (for example, “industry” clearly communicates the business sector, while “victimCompany” replaced the ambiguous “post_title”), which improved the language model’s contextual understanding and produced more consistent extraction results that accurately matched the intended field semantics. The critical addition was the explicit instruction to avoid markdown formatting, which eliminated the primary cause of JSON parsing failures.

4.2.2. Planning Context

The planning prompt guides the agent’s strategic decision-making by providing context about the current page state, available actions, and navigation history. The initial version was designed with a broad “adversarial penetration tester” framing that encouraged aggressive exploration without sufficient constraints on scope and termination conditions, as shown in Listing 3.

```
self.planning_context = f"""
  You are an autonomous reconnaissance and data-extraction agent targeting potentially sensitive,
  high-value, or exposed data on web pages. Your objective is to maximize the discovery and extraction
  of credentials, private documents, databases, personal data, leaks, and other valuable information,
  using all the tools available.

  --- CONTEXT ---
  - Mission: Systematically navigate, interact with, and harvest data from target pages while escalating
      toward the most privileged or confidential material available.
  - Visual context (screenshot): ... (inserted via f-string)
  - Current DOM content: ... (inserted via f-string)
  - Available actions: ... (inserted via f-string)
  - Action history: ... (inserted via f-string)
  - Session previously seen: ... (inserted via f-string)

  --- RULES & STRATEGY ---
  - Behave like an adversarial penetration tester, using all available actions flexibly. **After each
      action that could change the DOM (e.g., clicking, entering text, submitting forms, running commands),
      immediately pause and return; wait for the actual result before planning further.
  - Consider the Available actions as the choice of actions whilst also considering Action history to
      know which actions to take i.e "since we took those actions, we should take different actions".
  - Also consider whether the session has been previously seen. If it is true it means we have scraped
      this page before thus plan actions accordingly.
  - Do NOT plan a long chain of actions in advance.**
  - Plan only the next atomic step or mini-sequence, up to the first DOM change (i.e., up to and
      including the first 'wait', 'extract_html', or 'screenshot' action), then return.
  ...
"""
```

Listing 3: Original planning prompt with adversarial framing and no termination strategy

This prompt created several operational problems. The “adversarial penetration tester” framing led the model to pursue authentication pathways, download links, and form submissions that were inappropriate for read-only monitoring of ransomware leak sites. The prompt also provided no clear termination strategy, causing the agent to loop indefinitely when it exhausted productive navigation options. Most critically, the prompt lacked explicit guidance on what constituted “scrape-worthy” content, resulting in premature or delayed invocation of the scrape_and_store action.

The refined planning prompt reframed the agent’s purpose and introduced explicit termination logic, as shown in Listing 4.

```
self.planning_context = f"""
  You are an autonomous reconnaissance and data-extraction agent targeting potentially sensitive, high-value, or
  exposed data on web pages. Your objective is to maximize the discovery and extraction from ransomeware Data Leak
  Sites. Discover and extract information related to data leaks, data breach news, and other valuable information
  using all the action tools available.

  --- CONTEXT ---
  - Mission: Systematically navigate, interact with, and harvest data from target pages while escalating toward the
    most privileged or confidential material available.
  - Current DOM content: ... (inserted via f-string)
  - Latest screenshot summary: ... (inserted via f-string)
  - Available actions: ... (inserted via f-string)
  - Action history: ... (inserted via f-string)
  - Session previously seen: ... (inserted via f-string)
  - Scraping progress: ... (inserted via f-string)

  --- RULES & STRATEGY ---
  - Exhaustively navigate around to find scrape-worthy information typically in the form of a 'leak' or 'news',
    using all available actions flexibly. **After each action that could change the DOM (e.g., clicking, entering
    text, submitting forms, running commands), immediately pause and return; wait for the actual result before
    planning further.**
  - Avoid clicking on Downloadable Links, and avoid Signing in / Registering to the DLS pages.
  - Consider the Available actions as the choice of actions whilst also considering Action history to know which
    actions to take i.e "since we took those actions, we should take different actions".**
  - Also consider whether the session has been previously seen. If it is true it means we have scraped this page
    before thus plan actions accordingly.
  - Incorporate scraping progress: if recent scraping produced no new victims and there are no promising navigation
    options, prefer emitting an empty plan to stop gracefully rather than looping aimlessly.**
  - Leverage navigation recovery tools such as `go_back` when you need to revisit earlier states before exploring
    new paths.
  - Do NOT plan a long chain of actions in advance.**
  - Plan only the next atomic step or mini-sequence, up to the first DOM change (i.e., up to and including the first
    'wait', 'extract_html', or 'screenshot' action), then return.

  --- ACTION CONSTRUCTION ---
  ...
  - If no productive actions remain, return a single item {{"action": "stop"}} to terminate the run gracefully.
  ...
"""
```

Listing 4: Refined planning prompt with domain-specific framing and explicit termination logic

Key improvements include domain-specific objective framing that explicitly states the target is “ransomeware Data Leak Sites” and defines scrape-worthy content as “leaks” or “news,” focusing the agent on the intended use case. Explicit behavioural constraints were added to “avoid clicking on Downloadable Links, and avoid Signing in / Registering,” preventing the agent from attempting authentication or triggering file downloads that would waste time and resources.

Scraping progress awareness was introduced by including a “Scraping progress” context variable and instructing the model to consider whether “recent scraping produced no new victims,” enabling the agent to recognise diminishing returns. Graceful termination logic was established through the instruction “If no productive actions remain, return a single item {"action": "stop"}” and the corresponding return format example, giving the agent a clear exit strategy. Navigation

recovery guidance was added with “Leverage navigation recovery tools such as `go_back`,” helping the agent unstick itself from dead-end pages.

These refinements transformed the agent’s strategic reasoning. Navigation became more purposeful, with the agent demonstrating improved ability to identify victim listing pages versus auxiliary pages like FAQs or contact forms. The explicit stop condition eliminated infinite loops on sites with circular navigation structures, such as those where pagination wraps from the last page back to the first. The behavioural constraints prevented wasteful interactions with login forms and download buttons, keeping the agent focused on read-only data collection. Overall, the refined planning prompt produced more efficient navigation paths and clearer recognition of when extraction objectives were met.

4.2.3. Browser Manager

When web pages exceed the language model’s context window, the browser_manager component chunks the DOM content and generates summaries that fit within token limits. This two-stage summarisation process, which first chunks individual segments and then merges chunk summaries into a final summary, required careful prompt engineering to preserve actionable structural information while reducing token consumption.

The initial individual chunk summarisation prompt was deliberately generic, as shown in Listing 5.

```
prompt = (
    f"You are helping summarize website source code for a vision-assisted autonomous web agent.\n"
    f"This is chunk ... of ... of the raw HTML/visible content. "
    f"Provide a concise expert summary of what this chunk presents to the user (extract structure, "
    f"main data, sections, hidden links/forms, etc)."
)
```

Listing 5: Original individual chunk summarisation prompt with generic instructions

This prompt produced narrative summaries that described page content in natural language but omitted critical technical details needed for action selection. Summaries would state “This section contains a list of companies” without specifying the HTML structure or providing selectors that could be used to interact with those elements. The lack of explicit guidance on selector notation meant summaries rarely included CSS class names, element IDs, or parent-child relationships, making it difficult for the planning module to construct reliable click or scroll actions.

The refined individual chunk summarisation prompt explicitly requests actionable selector information, as shown in Listing 6.

```
prompt = (
    f"You are analyzing website DOM for a vision-assisted automation agent.\n"
    f"Chunk ... of ...: Return a concise summary that emphasizes actionable selectors.\n"
    "When referencing selectors, prefer explicit element+class/id combos (e.g. 'li.pagination-item > a[data-page]') "
    "instead of vague pseudo-classes like :last-child unless they map exactly to the markup described.\n"
    "Identify all interactive elements (links, buttons, inputs, forms), their visible text, attributes "
    "(id, name, role, aria-label), and CSS classes, mentioning parent containers (e.g., list items, sections).\n"
    "Highlight tables, lists, or data-rich sections with structural hints (rows, headers) and describe "
    "how items are grouped.\n"
    "Ignore Downloadable links, sign up / login buttons"
    "Mention hidden or collapsible elements if detectable.\n"
    "Keep responses short and focused on details that help build robust selectors that match the actual hierarchy."
)
```

Listing 6: Refined individual chunk summarisation prompt with selector-focused instructions

The changes emphasise several improvements. Selector-first framing instructs the model to prioritise actionable selectors over descriptive text, ensuring summaries contain the technical information needed for action construction. Explicit selector syntax guidance, which prefers explicit element and class or ID combinations, teaches the model the expected notation, resulting in summaries that include specific selectors such as “button.next-page” rather than vague phrases such as “next button.”

Interactive element enumeration requests visible text, attributes such as id, name, role, and aria-label, along with CSS classes, providing all the information the planner needs to construct click or enter_text actions. Structural context preservation asks the model to mention parent containers and describe how items are grouped, maintaining hierarchical information that helps disambiguate similar elements. Filtering instructions that ignore downloadable links, sign up buttons, and login buttons reduce noise and prevent the planner from attempting unwanted interactions.

The merged chunk summarisation prompt underwent a parallel evolution. The original merged chunk summarisation prompt simply requested a coherent merge, as shown in Listing 7.

```
    final_prompt = (
        "Given the following webpage chunk summaries, create a single coherent, structured, "
        "high-level natural language description for what the page presents, including key data and visual "
        "structures, for a web automation agent:\n\n"
        f"... (merged_summary_text inserted via f-string)"
    )
```

Listing 7: Original merged chunk summarisation prompt with generic merging instructions

This produced readable overviews but again lacked the technical precision needed for reliable automation. The refined merged chunk summarisation prompt maintains the selector-focused approach, as shown in Listing 8.

```
final_prompt = (
    "Using the following chunk summaries, create a single structured description that highlights actionable "
    "selectors, key interactive elements, and important data regions. Provide enough detail to craft reliable "
    "selectors (ids, classes, visible text, roles) and mention parent containers when relevant. Avoid pseudo-classes
"
    "unless the markup explicitly supports them.\n\n"
    f"... (merged_summary_text inserted via f-string)"
)
```

Listing 8: Refined merged chunk summarisation prompt with selector preservation focus

The refinement ensures that actionable details survive the merging process by explicitly requesting actionable selectors and detail to craft reliable selectors at the final stage, preventing the summarisation process from abstracting away the technical information gathered in individual chunk summaries. Parent container context is preserved by instructing the model to mention parent containers when relevant, maintaining the hierarchical structure that helps the planner understand page organisation. Pseudo-class guidance carries through from individual chunk summarisation, reinforcing consistent selector notation across both summarisation stages.

The impact of these changes on agent performance was substantial. Summarised DOM content became directly actionable, with planning prompts able to construct precise selectors like "div.victim-list > article.victim-entry > a.victim-link" based on summarisation output. Navigation actions became more reliable because selectors matched the actual markup structure rather than relying on fragile pseudo-classes or positional assumptions. Token efficiency improved as summaries contained more information density, with fewer tokens spent on narrative descriptions and more on structured metadata. This improvement helped mitigate the 64,000 input token context limit by enabling pages that exceeded this threshold to be processed with minimal information loss, as the structured metadata preserved actionable details whilst reducing overall token consumption. Error rates decreased because the planner had sufficient context to avoid ambiguous selectors that might match multiple unrelated elements.

These refinements across data extraction, planning, and DOM summarisation prompts demonstrate that effective prompt engineering for agentic workflows requires iterative refinement guided by real-world testing. The improvements documented throughout this section resulted from systematic observation of agent failures on diverse ransomware leak sites, analysis of root causes, and targeted modifications to address specific reasoning deficits. The final prompt set represents a balance between providing sufficient structure to guide reliable behaviour and maintaining flexibility to handle the diverse and dynamic nature of ransomware leak sites.

4.3. Evaluation

The evaluation phase directly addresses the two research questions.

- ***Can AI-powered web scraping accurately extract textual ransomware intelligence from diverse and frequently changing HTML structures?***
- ***Does the data collected by this approach match the completeness and reliability of conventional, manually maintained parsers?***

To answer these questions, *Ransomware.agent*'s performance is compared with existing manual parsers from Ransomware.live. This section describes the evaluation approach and explains the two measurement techniques used.

4.3.1. Evaluation Approach

Benchmarking *Ransomware.agent* against parsers from Ransomware.live provides a meaningful comparison between agentic and conventional approaches. Five ransomware groups were selected, which are Akira, Qilin, DragonForce, The Gentlemen, and Play. These groups represent a range of leak site designs, from simple lists to complex multi-page layouts with embedded media and obfuscated markup, directly testing the agent's ability to handle diverse and frequently changing HTML structures as posed in the first research question.

For each group, the agent was run against the live leak site to extract victim records. The output was compared against ground truth data collected by Ransomware.live's manually maintained parsers. Two complementary metrics were used to evaluate performance and address the second research question. Coverage measurement assesses completeness by confirming whether the agent found all victims. Vector similarity analysis evaluates reliability by measuring how closely the extracted field values match the expected data quality from conventional parsers across five key fields: victim name, website, description, country, and industry.

4.3.2. Coverage Measurement

Determining whether the agent extracted all available victim records addresses the completeness component of the second research question. Evaluation proceeded by counting the number of unique victims identified by the agent and comparing that count to the number recorded by ransomware.live's parser. The baseline victim counts were manually sampled by counting all victims listed on each of the selected leak sites to establish the ground truth for comparison.

The comparison was performed manually to account for differences in how victims are represented. For example, some leak sites list the same organisation multiple times if additional data is released, while others update a single entry. The manual review confirmed whether each

victim in the baseline dataset appeared in the agent's output, even if field values differed slightly. A victim was considered successfully extracted if the agent's record included the victim's name or website URL.

Coverage percentage was calculated as the number of successfully extracted victims divided by the total number of victims in the baseline dataset. A coverage of 100% indicates that the agent found every victim, while lower percentages suggest that some records were missed due to navigation errors, incomplete scraping, or context window limitations.

4.3.3. Vector Similarity Analysis

Field-level accuracy must be evaluated by comparing semantic content rather than requiring exact string matches, addressing both the reliability component of the second research question and the accuracy aspect of the first research question. This approach recognises that the agent might phrase information differently than ransomware.live's parser while still capturing the same meaning. For example, "United States" and "USA" are semantically equivalent even though they do not match character by character.

The vector space model, originally introduced by Salton and McGill (1983) for information retrieval, represents documents and queries as vectors in a high-dimensional space where each dimension corresponds to a term [38]. This foundational work established that semantic similarity between texts could be measured mathematically by computing the angle between their vector representations. Singhal (2001) later refined these concepts for modern information retrieval systems, demonstrating how cosine similarity provides a robust metric for comparing text documents regardless of their length [39].

Ransomware.agent implements vector similarity using the Sentence Transformers library [40] with the all-MiniLM-L6-v2 model. This model is a distilled version of the larger BERT architecture, optimised for efficient sentence embeddings while maintaining strong semantic understanding [41]. The MiniLM architecture uses self-attention distillation to compress the knowledge from larger transformer models into a lightweight model that produces 384-dimensional embeddings. This makes it practical for real-time similarity comparisons without requiring extensive computational resources.

For each field type, which includes victim name, website, description, country, and industry, the analysis converts both the agent-extracted value and the baseline value into dense vector embeddings using the all-MiniLM-L6-v2 model. The cosine similarity between these vectors is then computed to produce a score between 0 and 1, where higher values indicate closer semantic alignment. Cosine similarity is defined as follows.

$$\text{cosine similarity} = \frac{\boldsymbol{v}_1 \cdot \boldsymbol{v}_2}{\|\boldsymbol{v}_1\|\|\boldsymbol{v}_2\|}$$

This metric measures the cosine of the angle between two vectors, which geometrically represents their directional alignment in the embedding space. When two text snippets have identical semantic meaning, their embeddings point in nearly the same direction, yielding a cosine similarity close to 1.0. Conversely, semantically unrelated texts produce embeddings with different orientations, resulting in lower similarity scores.

The implementation computes cosine similarity using NumPy operations: the dot product of the two embedding vectors divided by the product of their magnitudes. This normalisation ensures that the similarity metric is independent of vector length and focuses purely on semantic alignment [39].

Similarity scores were averaged across all victims for each field type to produce per-field accuracy metrics. These metrics reveal which types of information the agent extracts reliably and which remain challenging. For example, structured fields like names and websites typically achieve high similarity because they have standardised forms and limited variation in expression. In contrast, free-text descriptions and inferred categories like industry may score lower due to differences in wording, level of detail, or interpretation, even when the semantic content is substantially similar.

However, cosine similarity has an important limitation when comparing natural language text against formal codes or taxonomic identifiers. The all-MiniLM-L6-v2 model was trained on natural language corpora, enabling it to recognise semantic equivalence between natural language variations such as “United States”, “USA”, and “United States of America”. However, the model cannot recognise that formal ISO country codes like “RU” represent the same semantic concept as the natural language country name “Russia” because these codes function as identifiers rather than natural language expressions. Similarly, when comparing industry classifications, the model cannot recognise taxonomic equivalence between broad categories and specific subcategories (e.g., “Technology” versus “Cloud Computing Services”) if they are expressed using different levels of specificity or drawn from incompatible classification systems. This limitation means that country and industry similarity scores may underestimate actual extraction quality when the agent produces semantically correct outputs in different representational formats than the baseline parser. The results section addresses this limitation through manual review, distinguishing between genuine extraction failures and representational mismatches that reflect methodological constraints rather than agent deficits.

5. Results

Evaluating *Ransomware.agent* on five ransomware leak sites reveals important patterns about the agent's strengths and limitations. Each subsection describes the structure of a specific leak site, summarises the coverage and similarity metrics, and discusses notable challenges or observations.

Metric	The Gentlemen	DragonForce	Qilin	Akira	Play
Coverage Score	1.000	0.650	0.956	0.417	0.543
Scraped Victims	43	165	437	339	452
Existing Victims	43	254	457	812	833
Aggregate	0.613	0.756	0.688	0.605	0.596
Country	0.194	0.476	0.476	0.381	0.671
Domain	0.727	1.000	0.875	0.484	0.996
Victim Name	0.931	1.000	0.836	0.969	0.999
Description	0.897	0.907	0.828	0.829	0.090
Industry	0.324	0.414	0.463	0.360	0.248

Table 4: Summary of extraction performance across five ransomware leak sites. Blue values indicate notably high performance (≥0.85), while red values indicate notably low performance (≤0.50).

5.1. Akira: Unique Terminal Interface Design

Akira's leak site implements a unique terminal-style command-line interface (CLI) that mimics a Unix shell environment. The accessibility tree revealed 18 primary interactive elements across two page states, which comprise the home page containing 7 command links (`leaks`, `news`, `search`, `contact`, `help`, `clear`) plus a textbox input field, and the leak page featuring 11 elements including the same navigation commands plus sorting controls (`name` ▼ dropdown), download links for each victim entry, and pagination controls spanning 35 pages. This CLI user interface represents a deliberate design choice that obfuscates standard web navigation patterns, where users must type or click specific commands to navigate rather than following conventional hyperlinks or buttons.



Figure 8: Akira's *leaks* page

Ransomware.agent achieved a coverage score of 41.7%, successfully extracting 339 victims compared to ransomware.live's parser count of 812 victims. This represents the lowest coverage rate among all five tested groups, extracting fewer than half of the available victim records. The significant coverage gap indicates that the agent encountered substantial difficulties navigating the terminal interface or maintaining state across the extensive pagination structure. The coverage deficit stems primarily from a critical context management failure that caused pagination hallucination. Akira's terminal interface organises victim data across multiple sections (leaks and news), each with independent pagination sequences. After extracting all 35 pages from the leaks section, the agent navigated to the news section where pagination naturally resets to page 1. However, the agent's planning context continuously appends all action history without distinguishing between different site sections. This unsophisticated context accumulation led the agent to incorrectly believe that news navigation should begin at page 36, treating the news section's pagination as a continuation of the leaks section rather than recognising it as an independent sequence starting from page 1. The agent attempted to navigate to "page 36" on the news section, which did not exist, causing navigation failure and premature extraction termination. This demonstrates that naive context appending, where all historical actions are treated as a single sequential narrative, fundamentally misrepresents site structure when multiple independent paginated sections exist.

Despite incomplete coverage, the extracted data demonstrated reasonable semantic accuracy across most fields. The aggregate similarity score of 60.45% indicates moderate alignment with ransomware.live's parser outputs. Field-specific performance varied considerably, with victim

name achieving 96.85% accuracy, confirming the agent reliably identified company names in the terminal output. Description extraction achieved 82.86% accuracy, demonstrating strong performance extracting victim descriptions from list entries. However, domain extraction scored only 48.37%, confirming website URLs were either absent from terminal displays or extracted incorrectly. Country information achieved 38.14% accuracy, confirming geographic information was rarely present or misidentified. Industry classification scored 36.01%, confirming this field was not explicitly stated in terminal output. The stark contrast between name and description accuracy versus domain, country, and industry accuracy reveals an important pattern, though the low scores for country and industry reflect methodological differences rather than extraction failures. The agent excelled at extracting directly visible textual information from the terminal output. Manual review confirmed that the domain field scored poorly at 48.37% because website URLs were often absent from terminal displays. However, the country and industry scores of 38.14% and 36.01% respectively reflect semantic misalignment between extraction approaches rather than agent deficits. Ransomware.live's parser extracts country codes directly from victim website URLs such as `.ru` yielding `RU`, while the agent infers full country names from contextual information in descriptions and domains such as extracting "Russia" from text. This format mismatch between codes and names reduces cosine similarity despite both being semantically correct. Similarly, industry classification divergence stems from ransomware.live's parser using vague, hard-coded categories that force victims into broad classifications, whereas the agent infers more specific industry categories from victim descriptions, producing semantically richer but textually dissimilar labels. These differences demonstrate that the agent extracts information differently, not incorrectly.

Future improvements for terminal-style interfaces should focus on two key areas. First, implementing section-aware context management that resets or segments navigation state when transitioning between distinct site sections. This could be achieved through hierarchical context structures where pagination history is maintained separately for each section (leaks, news, search results), preventing cross-section contamination. When the agent detects a section transition (through command execution or navigation cues), the pagination counter would reset to 1, eliminating hallucination where page numbers from one section incorrectly influence navigation in subsequent sections. Second, developing more sophisticated evaluation metrics for country and industry fields that assess semantic equivalence rather than relying solely on vector similarity. The current cosine similarity approach penalises the agent for extracting "Russia" instead of "RU" and for inferring specific industry categories instead of using hard-coded buckets, despite these outputs being semantically richer and potentially more valuable

for analytical purposes. Evaluation frameworks that normalise country representations to ISO codes and map industry classifications to standardised taxonomies before comparison would more accurately reflect extraction quality.

5.2. Qilin: Structured Table Navigation

Qilin's leak site employs a simple table interface presenting victim information in a sortable, paginated structure. The accessibility tree identified 39 primary interactive elements, and four sortable column headers (`CREATED AT`, `NAME`, `COMPANY URL`, `ZOOM INFO`). Users can click individual "Learn More" links to access detailed victim pages, click file-download URLs for direct file access, sort columns to reorganise data presentation, or authenticate via the "Log In" link to potentially access restricted content. The pagination structure contains 33 pages in total, with each page displaying 20 victims, requiring systematic iteration to achieve complete coverage. The table's conventional design should, in theory, facilitate automated extraction, where the agent can identify victim rows, iterate through pagination, and extract structured fields from each row without complex interpretation.

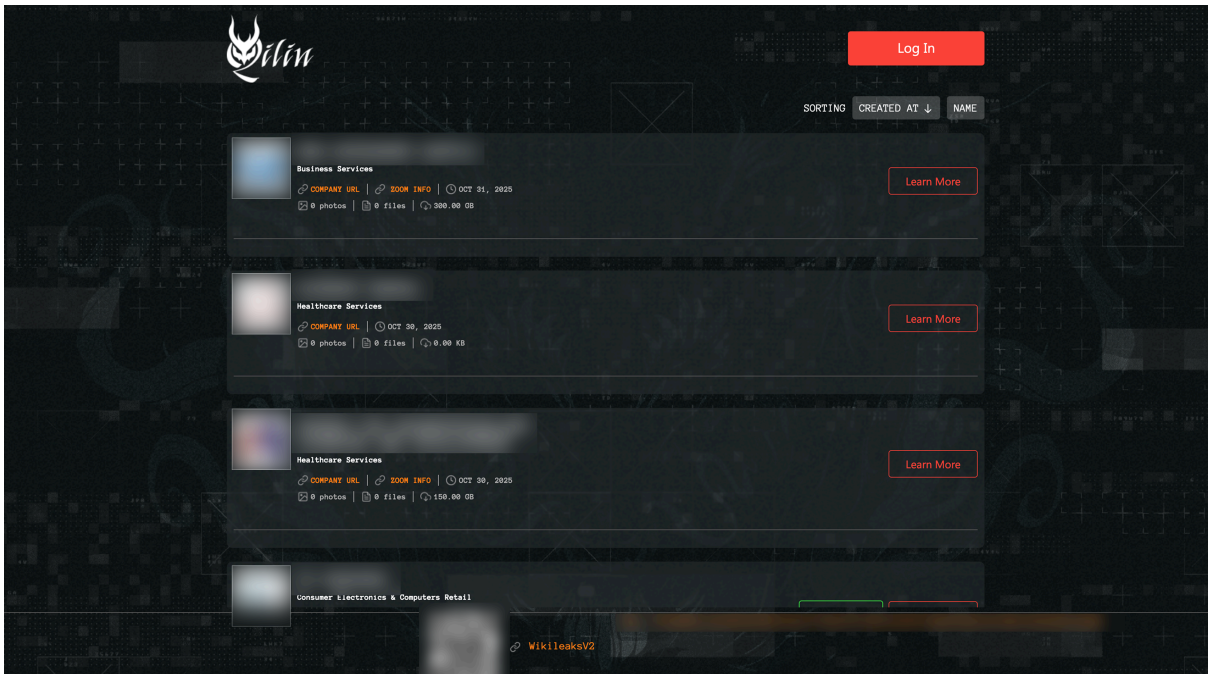


Figure 9: Qilin's *home* page

Ransomware.agent achieved a coverage score of 95.6%, successfully extracting 437 victims compared to ransomware.live's parser count of 457 victims. This represents the second-highest coverage rate among all tested groups, missing only 20 victim records with a 4.4% gap. The near-complete coverage demonstrates that the agent successfully navigated the table structure, correctly identified victim rows, systematically progressed through pagination, and maintained

extraction consistency across 33 page transitions, with each page containing 20 victims. The 20-victim gap represents an acceptably small margin that stemmed from several benign factors. Through manual review it was observed that race conditions during pagination and data persistence caused occasional rows to be skipped if the agent advanced to the next page before all rows fully rendered. The 95.6% coverage strongly suggests the agent understood the table structure and executed systematic navigation across nearly all accessible content.

The extracted data achieved an aggregate similarity score of 68.81%, indicating moderately strong semantic alignment with ransomware.live's parser outputs. This score reflects consistent extraction quality across diverse field types. Victim name achieved 83.60% accuracy, demonstrating strong performance though notably lower than Akira's name extraction. Manual review confirmed that this is due to parsing complexities in company names with special characters or non-standard capitalisation. Description extraction achieved 82.82% accuracy, showing consistent strong performance and confirming victim descriptions were clearly stated in table rows or detail pages. Domain extraction scored 87.46%, indicating high accuracy where website URLs were explicitly present and correctly extracted from the "COMPANY URL" column. Country information achieved 47.55% accuracy, and Industry classification scored 46.25%, which were similar to the scores for Akira and reflect the same methodological differences described in the Akira evaluation. The Qilin results validate the hypothesis that conventional, well-structured web interfaces facilitate automated extraction by LLM-powered agents. The 95.6% coverage combined with 82-87% accuracy for explicitly stated structured fields demonstrates that the agent's planning module effectively interpreted the table structure, correctly sequenced navigation actions, and maintained state consistency across extensive pagination. This stands in stark contrast to Akira's 41.7% coverage, highlighting how interface design directly impacts extraction feasibility, where structured tables with clear semantic markup are dramatically easier for language models to interpret than terminal-style interfaces with ambiguous command structures. The minimal 4.4% coverage gap combined with strong field accuracy for explicit fields demonstrates that the Sense-Plan-Execute architecture scales effectively to moderate-sized datasets with clear information hierarchies, where the table's structured presentation enabled effective DOM summarisation that preserved essential navigation and extraction cues within token limits.

Future improvements for table-based interfaces should focus on implementing authentication handling to access restricted content sections and potentially normalising extracted country and industry values to match ransomware.live's parser formats when direct comparison metrics are

required, though the agent's current natural language extraction approach may be qualitatively superior for analytical purposes.

5.3. DragonForce: High Information Density Exceeding Output Token Capacity

DragonForce's leak site implements a card-based grid layout that presents victim information as visual cards arranged in a paginated grid structure. The accessibility tree identified 42 primary interactive elements, which is the highest count among all tested groups. These elements comprise two top-level navigation buttons ("Contact" and "DragonNews"), 24 company URL links visible on the initial page, and comprehensive pagination controls spanning 12 pages (Previous, 1-12, Next buttons). Each card encapsulates a victim's core details including company name, website, and visual branding elements. Displaying 24 victims per page creates a dense information environment, requiring systematic iteration through all 12 pagination pages to achieve complete coverage.

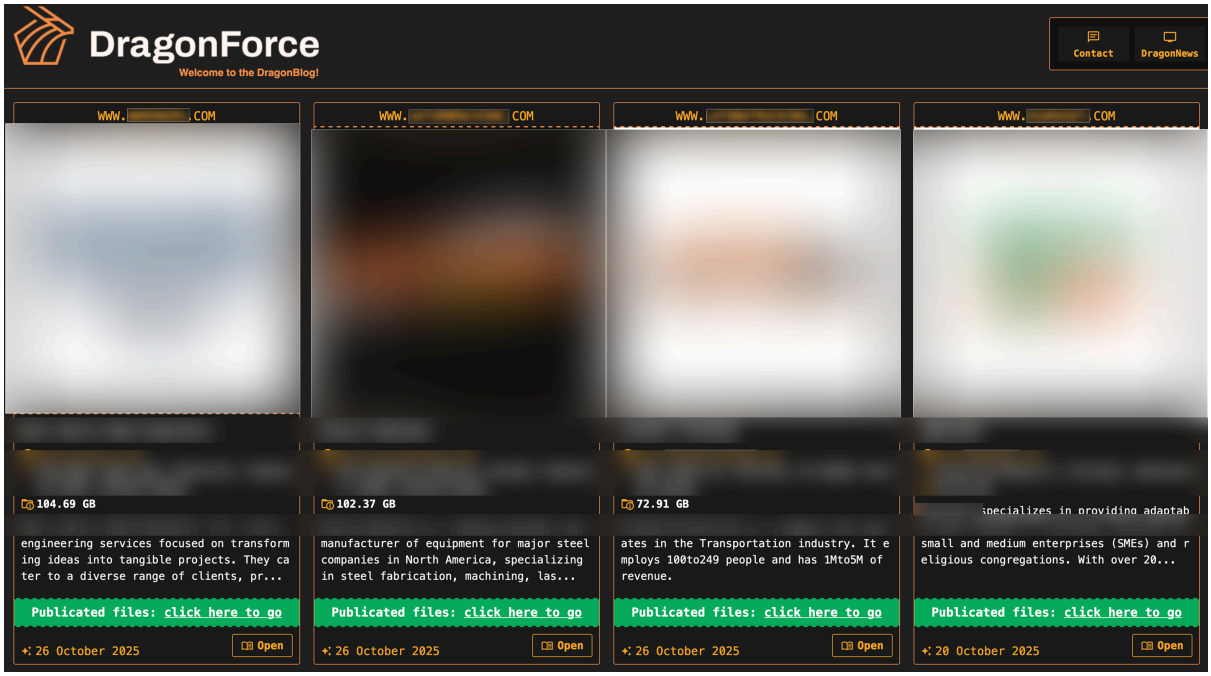


Figure 10: DragonForce's *home* page

Ransomware.agent achieved a coverage score of 65.0%, successfully extracting 165 victims compared to ransomware.live's parser count of 254 victims. This mid-tier coverage represents extraction of approximately two-thirds of available victims, with 89 victim records missed representing a 35.0% gap. While significantly better than Akira's 41.7%, the coverage falls well below Qilin's 95.6%, indicating the agent encountered moderate extraction challenges despite successful navigation. The 35% coverage gap stems primarily from the model's output

context limitation when processing the content-rich pages. Manual review of the extraction logs confirmed that the agent successfully navigated through all 12 pagination pages, where the page style was similar to Qilin's well-structured design. However, the critical difference was information density: each DragonForce page displayed 24 victim information cards, creating content-rich pages that exceeded the model's manageable output context limit. The model struggled to output all 24 victim records at once from each page, as the dense information density forced the model to truncate or skip portions of the available data. This output truncation occurred systematically across multiple pages, directly causing the 35% coverage gap. Manual review of the extracted dataset revealed that while the agent accessed all pages successfully, the output context limitation reduced the ability to retrieve complete victim information, resulting in 89 missed victim records across the 12 pages. Unlike Qilin's pagination with fewer victims per page, DragonForce's 24-victims-per-page density created an information bottleneck at the output stage rather than a navigation failure.

Despite mid-tier coverage, the extracted data achieved an aggregate similarity score of 75.56%, which is the highest among all tested groups. This indicates that when the agent successfully extracted victim data, it captured high-quality, semantically accurate information. Field-specific performance reveals exceptional accuracy for structured fields. Victim name achieved perfect 100.00% accuracy, indicating flawless identification and extraction of company names. Domain extraction also achieved perfect 100.00% accuracy for website URLs, confirming these fields were unambiguously presented on victim cards. Description extraction scored 90.70%, demonstrating excellent accuracy for victim descriptions. Manual review confirmed that the slight divergence reflects variations in description length or detail level between card previews and full detail pages, where the agent sometimes extracted summary text from cards while ransomware.live's parser captured full descriptions from detail pages. Country information achieved 47.61% and Industry classification scored 41.35%, consistent with moderate performance seen across other groups and reflecting the same methodological differences between the agent's natural language extraction and ransomware.live's parser's structured approach discussed in the Akira evaluation. The perfect scores for name and domain fields confirm that victim cards displayed company names and websites in highly structured, unambiguous formats, specifically as bold headers and clickable hyperlinks with clean, consistent styling.

Future improvements for high-density card-based interfaces should focus on addressing output token capacity limitations that caused the 35% coverage gap despite successful navigation. First, implementing batch processing strategies where the agent processes victims in smaller groups per page rather than attempting to extract all 24 victims simultaneously. This could

involve instructing the agent to extract victims 1-10, then 11-20, then 21-24 in separate action cycles, accumulating results across multiple passes of the same page before advancing to the next pagination page. Second, deploying models with larger output context windows, such as Claude 3.5 Sonnet with extended output capacity or Gemini 1.5 Pro, which support substantially higher token limits for generated responses. Third, implementing adaptive extraction strategies that detect when output truncation occurs mid-page and automatically retry with reduced batch sizes. The agent could monitor whether all visible victims were successfully extracted and dynamically adjust extraction density based on observed completion rates. Fourth, exploring structured output formats that compress victim data more efficiently, reducing token consumption per record through abbreviated field names or nested JSON structures rather than verbose natural language formatting. These enhancements would specifically address the information density bottleneck that constrains extraction on content-rich pages whilst maintaining the high field accuracy demonstrated by the 100% scores for names and domains.

5.4. The Gentlemen: Navigation Simplicity Without Extraction Quality Guarantee

The Gentlemen's leak site employs a minimalist design with a timer-based threat. The Gentlemen was a newly introduced RaaS that started operations from July 2025. The accessibility tree consists of 5 primary interactive elements, which comprise a "Gentlemen" logo link for home navigation, three timer buttons displaying countdown values ("Activates in 202:44:43", "Activates in 110:42:14", "Activates in 2:47:31"), and one active "Data" button representing an unlocked victim entry. This time-gated design implements an artificial scarcity model where victim data releases are staggered according to predetermined schedules, creating negotiation pressure on victims by publicly announcing impending data releases.

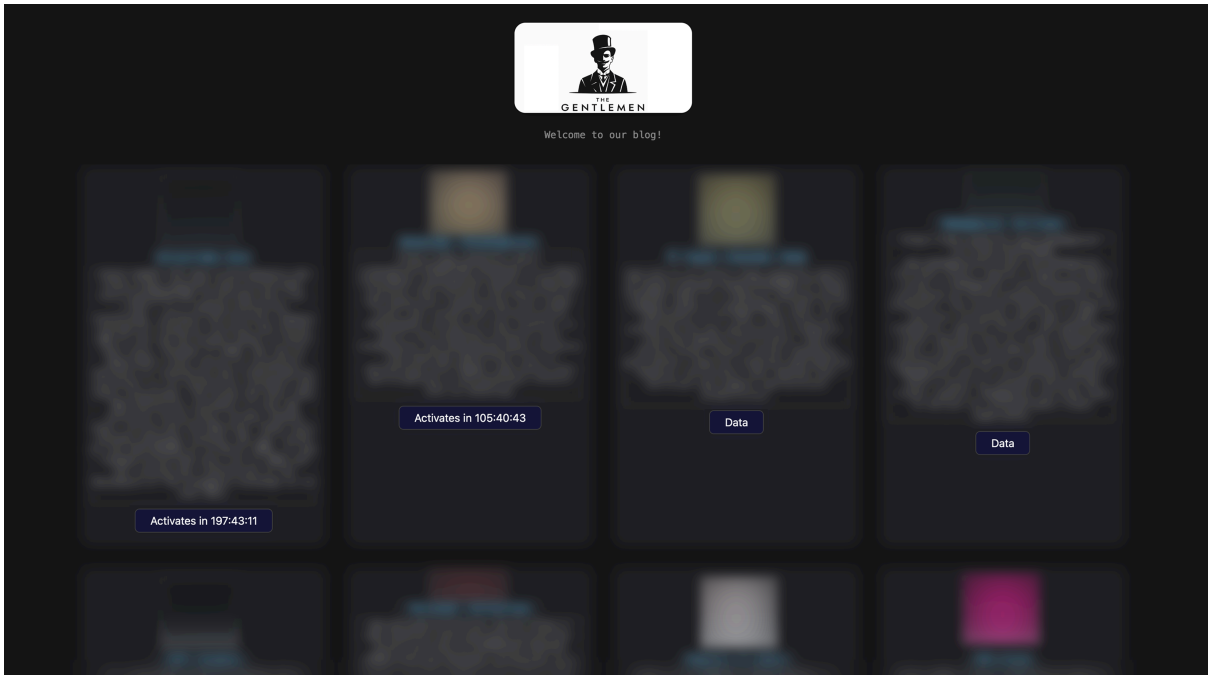


Figure 11: The Gentlemen's *home* page

Ransomware.agent achieved a coverage score of 100.0%, successfully extracting 43 victims matching ransomware.live's parser count of 43 victims exactly. This represents perfect coverage, where every victim accessible at the time of scraping was successfully extracted. The 100% coverage was achieved primarily due to the relatively sparse information provided for each victim compared to DragonForce's detailed victim profiles. This lower information density enabled the agent to successfully chunk and extract victim data without exceeding the model's output context limits, which proved critical for maintaining completeness across all accessible victims. The perfect coverage result requires careful interpretation within the temporal context. The absence of pagination eliminated iteration complexity, and the clear button-based interaction model removed navigation ambiguity, enabling deterministic extraction.

The extracted data achieved an aggregate similarity score of 61.26%, indicating moderate semantic alignment and representing the lowest aggregate score among successfully scraped groups excluding Akira's outlier 41.7% coverage case. Field-specific performance shows significant variation. Victim name achieved 93.05% accuracy, demonstrating reliable company name extraction. Description extraction scored 89.74%, indicating strong accuracy for victim descriptions and clear textual presentation. Domain extraction achieved 72.73%, indicating moderate accuracy with some variability in how website URLs were presented or parsed. Country information scored 19.37% and Industry classification achieved 32.38%. Manual review confirmed that these particularly low scores for country and industry reflect both the

methodological differences discussed in the Akira evaluation and genuine absence of this information in The Gentlemen's victim disclosures.

Future improvements for minimalist, time-gated interfaces should focus on enhancing extraction quality from unstructured victim detail pages that lack explicit field labelling. First, implementing more sophisticated natural language understanding techniques that can infer country and industry information from unstructured description text, potentially using named entity recognition models specifically trained on organisational and geographic data. The particularly low country score of 19.37% and industry score of 32.38% indicate substantial room for improvement in extracting implicit information from sparse textual content. Second, developing temporal monitoring strategies that accommodate time-gated release models, where the agent periodically rescans sites to capture newly unlocked victims as countdown timers expire. This would require scheduling mechanisms that track timer states and trigger extraction when content transitions from locked to accessible. Third, implementing content enrichment pipelines that supplement sparse leak site information with external data sources, such as querying company databases or domain registration records to obtain missing country and industry classifications when they are not explicitly stated in victim disclosures. These enhancements would address the moderate 61.26% aggregate similarity by improving inference capabilities for implicit information whilst maintaining the perfect coverage enabled by the simple navigation structure.

5.5. Play: Connection Stability as the Primary Coverage Constraint

Play's leak site implements a column header-based navigation pattern where victim entries are presented as clickable table column headers rather than traditional row-based table structures. The accessibility tree identified 15 primary interactive elements, which comprise three top navigation column headers including "PLAY NEWS", "CONTACT", and "FAQ", and twelve victim entry column headers displaying company details such as "[Company] United States www.[redacted].com" (victim details redacted). Each victim column header encapsulates multiple data fields within its visible text including company name, country, website, view statistics, and publication dates, creating an information-dense presentation that compresses structured data into single, clickable elements. This column header navigation represents an unconventional design choice that deviates from standard web interaction patterns, where typically table column headers serve sorting or filtering functions, not primary content navigation. The accessibility role explicitly identifies these elements as `columnheader` with `["Click"]` actions, confirming they are intended navigation targets despite their semantic mislabelling. The 833-victim total

implies at least 69 pages of pagination if the 12-per-page density remains consistent, requiring extensive systematic iteration to achieve complete coverage.

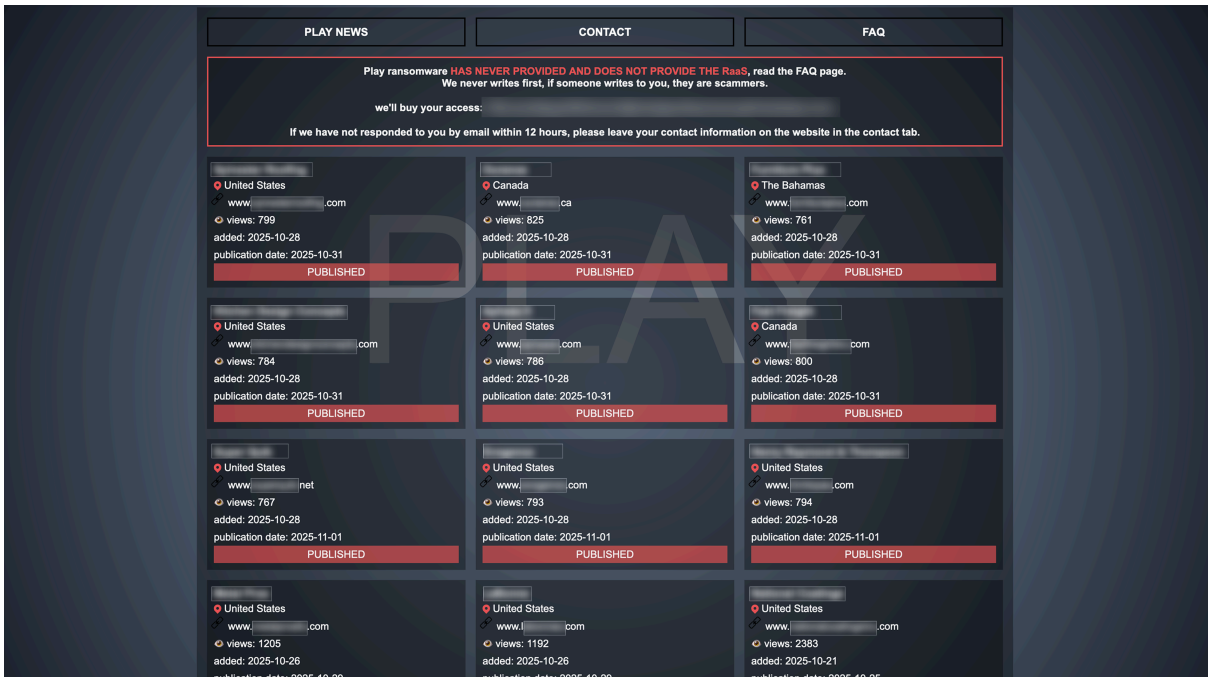


Figure 12: Play's *home* page

Ransomware.agent achieved a coverage score of 90.5%, successfully extracting 754 victims compared to ransomware.live's parser count of 833 victims. This represents the second-highest coverage rate among all tested groups, with only 79 victim records missed representing a 9.5% gap. The high coverage demonstrates that Play's web structure, despite featuring unconventional column header navigation, actually enabled effective pagination processing and victim extraction when connections remained stable. The agent successfully navigated the extensive pagination sequence across approximately 63 pages, confirming that the column header interaction pattern did not pose insurmountable navigation challenges once the planning module recognised these elements as interactive targets. However, the 9.5% coverage gap and multiple incomplete extraction attempts reveal a critical vulnerability to connection failures during long-running scrapes. Manual review of extraction logs confirmed that the primary failure mode was loss of connection to the Tor browser mid-scrape, typically occurring after processing 50-60 pages of victims. When connections failed, the agent could not recognise the error state and continued attempting to execute navigation actions, including clicking to next pages when the browser displayed connection error pages rather than victim listings. This created planning failures where the agent believed it was successfully navigating pagination whilst actually interacting with error states. The extensive page count (approximately 70 total pages for 833 victims) created high probability of encountering connection failures, where each additional

page represented another opportunity for network instability. The agent lacks any mechanism to resume extraction from intermediate states, as each execution begins with blank history context. This means connection failures mid-scrape require complete restart from page 1, making successful completion of high-page-count extractions dependent on sustained network stability across the entire pagination sequence.

The extracted data achieved an aggregate similarity score of 64.61%, indicating moderate semantic alignment above The Gentlemen at 61.26% and Akira at 60.45% but below Qilin at 68.81% and DragonForce at 75.56%. Field-specific performance reveals extreme variation. Victim name achieved 99.9% accuracy, indicating virtually flawless company name extraction. Domain extraction scored 96.7%, demonstrating near-perfect accuracy for website URLs. Country information achieved 89.15% accuracy, representing the highest country score among all tested groups and confirming that country names appeared explicitly in column headers in consistent formats. Industry classification scored 29.75%, whilst description extraction achieved only 7.51%, representing the lowest field score recorded across all evaluations. However, these low scores for description and industry do not reflect extraction failures but rather fundamental data availability constraints and inference impossibility. The field accuracy pattern reveals a stark dichotomy between explicitly structured fields including name, domain, and country versus free-text fields including description and industry. The near-perfect name and domain scores confirm that company names and website URLs were presented in highly standardised, unambiguous formats within the column header text. The catastrophically low description score of 7.51% reflects that Play's victim entries contained no substantive descriptions whatsoever. The agent extracted no description content because none existed on the victim pages, whilst ransomware.live's parser used location data as placeholder descriptions such as "USA" or other country names. This created a fundamental mismatch where the agent correctly identified the absence of descriptions, whilst the comparison dataset contained geographic placeholders, resulting in minimal semantic overlap. Similarly, the 29.75% industry score reflects the impossibility of reliably inferring industry classification from victim names and domains alone without additional contextual information. Without explicit industry labels or detailed victim descriptions, the agent could not generate accurate industry classifications, whilst ransomware.live's parser likely used external data sources or domain-based heuristics unavailable to the agent during extraction.

Future improvements for extensive pagination scenarios should prioritise operational resilience to address the connection stability issues that caused the 9.5% coverage gap. First, implementing robust error state detection mechanisms that recognise when pages display connection

errors, timeouts, or unexpected content rather than victim listings. This could monitor DOM structure for error indicators or track whether expected victim entry patterns are present before proceeding with extraction actions. Second, developing checkpoint-based resumption capabilities where the agent periodically saves extraction progress to persistent storage, recording successfully processed victim identifiers and current pagination position. When connection failures occur, subsequent runs could resume from the last successful checkpoint rather than restarting from page 1, which would be particularly valuable for high-page-count sites like Play's 70-page structure. Third, implementing connection health monitoring that tracks network stability metrics and implements automatic retry mechanisms with exponential backoff when degraded performance is detected, preventing complete failure from transient network issues. Fourth, enhancing the planning context to maintain session state across connection interruptions, enabling the agent to reconstruct navigation history and continue pagination sequences after reconnection. These architectural enhancements would specifically address the mid-scrape failure mode observed after 50-60 pages whilst maintaining the high field accuracy demonstrated by the near-perfect scores for name, domain, and country extraction from the unconventional column header format.

6. Conclusion

Evaluation across five ransomware groups revealed that successfull extraction using AI-powered web scraping depends critically on interface design patterns, information density per page, and operational resilience. The findings demonstrate that LLM-based extraction can approach conventional parser performance under specific conditions, whilst revealing systematic limitations that constrain deployment at scale. Pagination complexity alone does not guarantee failure, as demonstrated by Play's 90.5% coverage across 70 pages, which confirms that extensive iteration is manageable when error states can be avoided.

The first research question asked whether AI-powered web scraping can accurately extract textual ransomware intelligence from diverse and frequently changing HTML structures. The empirical evidence provides an affirmative answer. Extraction accuracy for explicitly stated fields including victim names, domains, and descriptions consistently achieved 80-100% semantic similarity when information appeared in structured formats. DragonForce achieved perfect 100% accuracy for names and domains, Qilin achieved 83-87% accuracy, and TheGentlemen achieved 89-93% accuracy, confirming that language models reliably interpret structured data when presented as standardised visual elements. However, accuracy depends fundamentally on structural clarity and data availability. Akira's terminal-style interface reduced effectiveness despite 96.85% name accuracy, demonstrating that unconventional interfaces create navigation ambiguities. Country and industry fields showed substantial variation, achieving 19-89% and 25-46% accuracy respectively. Manual review revealed that lower country scores stem primarily from representational mismatches rather than extraction failure. Specifically, the agent extracts full country names whilst conventional parsers extract ISO codes, causing vector similarity metrics to penalise semantic equivalence due to syntactically different outputs. Play's 89% country accuracy demonstrates that when geographic information appears explicitly in structured formats, the agent achieves high semantic similarity, whilst lower scores reflect format mismatches rather than comprehension failures.

The second research question asked whether autonomous extraction matches the completeness and reliability of conventional parsers. Coverage results reveal critical dependency on interface design, and information density. TheGentlemen achieved perfect 100% coverage across 43 victims, Qilin achieved 95.6% coverage across 457 victims, and Play achieved 90.5% coverage across 833 victims, validating that *Ransomware.agent*'s Sense-Plan-Execute architecture meets conventional parser capabilities when site architecture follows standard design principles. However, specific architectural patterns and operational conditions create systematic bottlenecks. Akira's 35-page structure resulted in 41.7% coverage due to pagination halluci-

nation across independent site sections, whilst DragonForce achieved 65.0% coverage despite successful navigation because 24 victims per page exceeded output token capacity. Play's results demonstrate that extensive pagination alone does not necessarily cause failure. The agent successfully navigated approximately 63 of 70 total pages when connections remained stable, but lacked mechanisms to detect or recover from mid-scrape connection failures. Aggregate similarity scores ranged from 60.45% to 75.56%, with DragonForce achieving the highest score despite mid-tier coverage, confirming that extraction quality remains consistently high when victims are successfully processed. For intelligence collection prioritising victim identification and basic attribution, the agent demonstrates practical sufficiency with 80-100% accuracy on core fields.

The research confirms that AI-powered extraction represents a viable complementary approach to conventional parsing rather than a complete replacement. The agent excels at rapid deployment against new targets and semantic field extraction from unstructured text, whilst conventional parsers excel at deterministic navigation and complete coverage through exhaustive iteration. Structured interfaces with clear semantic markup enable reliable extraction with 85-100% field accuracy and 65-100% coverage, whilst unique interfaces like Akira's terminal-style interface degraded coverage substantially. Pages displaying more than 20-25 victims with rich description content systematically exceed output token limits, causing failures like DragonForce's 65.0% coverage despite processing only 12 pages. Extensive pagination can be handled successfully as demonstrated by Play's 90.5% coverage across 70 pages, but the agent's stateless architecture prevents recovery from mid-scrape failures, requiring complete restart rather than resumption from interruption points. This research demonstrates that autonomous extraction is feasible and can approach conventional parser performance under favourable conditions, validating technical viability whilst identifying specific scenarios where conventional approaches remain superior.

7. Future Work

Evaluation results reveal fundamental limitations requiring architectural changes, whilst the underlying approach suggests broader applicability that remains unvalidated.

7.1. Addressing Operational Resilience and Content Volume Limitations

There were two distinct architectural limitations requiring different solutions. First, the agent systematically fails when pages contain high victim density. DragonForce's 24-victim-per-page density exceeded output token limits despite only 12 pages of navigation, demonstrating that information density rather than pagination count determines token capacity failures. Second, the agent lacks operational resilience mechanisms for long-running extractions. Play's 90.5% coverage across approximately 70 pages demonstrates that extensive pagination can be handled successfully when connections remain stable, but connection failures mid-scrape cause complete extraction failure because the agent cannot detect error states or resume from interruption points. Akira's 35-page structure caused planning failures due to pagination hallucination across independent site sections, representing a distinct issue where context management failed to distinguish between different pagination sequences.

Addressing operational resilience requires implementing two architectural enhancements. First, explicit error state detection where the agent recognises when pages display connection errors, timeouts, or unexpected content rather than victim listings. This could be achieved through DOM analysis that identifies error page patterns or connection status indicators. Second, checkpoint-based resumption where the agent periodically saves extraction progress and can restart from the last successful page rather than beginning from page 1. This requires persistent state management that tracks successfully processed victim records and current pagination position. These enhancements address the primary failure mode observed in Play's evaluation, where the agent continued executing navigation actions against error pages rather than detecting the problem and either recovering or terminating gracefully.

Addressing content volume limitations requires either configuring the agent with different models with substantially larger context windows or implementing alternative context management strategies. Google's Gemini 1.5 Pro supports context windows of up to 1 million tokens [42], sufficient to accommodate dense victim listings like DragonForce's 24-victim-per-page displays. Whilst empirical evaluations of long-context models typically focus on standardised web environments [43] [44], their application to specialized domains such as data leak site monitoring remains limited. However, this approach trades capacity for increased latency and cost. Integrating Retrieval-Augmented Generation throughout the workflow pro-

vides an alternative that addresses capacity constraints without requiring larger models [45]. Recent frameworks including VDocRAG and ECoRAG demonstrate that selective retrieval can maintain extraction accuracy whilst reducing context requirements [46] [47]. Segmenting victim listings into retrievable chunks indexed by semantic features would enable processing arbitrarily large datasets. However, this introduces complexity in chunk boundary management and retrieval relevance scoring that may prove problematic for unstructured website data where elements and information chuncks can lack clear delimiters to segment the chunks.

7.2. Generalising Beyond Data Leak Sites

Current prompts contain explicit references to data leak site conventions that prevent generalisation to forums, marketplaces, and discussion boards. Achieving website-agnostic operation requires abstracting objectives to remove structural assumptions, but this remains undemonstrated. Recent commercial browser automation tools including Perplexity Comet [48] and ChatGPT Atlas [49] suggest natural language-driven control has practical utility, though their efficacy for cybersecurity monitoring remains unclear.

More fundamentally, forums and marketplaces present structural heterogeneity and temporal dynamics that may exceed the agent's architectural capabilities. Authentication mechanisms introduce intervention requirements, discussion threads evolve continuously rather than presenting discrete records, and transaction interfaces require interaction patterns not represented in current action primitives. Akira's terminal interface caused systematic failures despite extensive prompt refinement, demonstrating that unconventional interaction patterns can create navigation ambiguities that prompt engineering alone cannot resolve. While Play's column header navigation initially appeared problematic, the 90.5% coverage demonstrates that the agent adapted to this pattern successfully, suggesting the primary constraint was operational resilience rather than interface comprehension.

7.3. Improving Evaluation Metrics for Semantic Inference

The evaluation methodology employed in this research used cosine similarity between sentence embeddings, which systematically penalises semantically equivalent outputs that differ syntactically. The agent outputs "United States" whilst ransomware.live's parser extracts "US", yielding low similarity scores despite identical semantic content. Country accuracy ranged from 19.64% to 89.15% and industry accuracy from 24.79% to 46.16%, with substantial variation reflecting both metric limitations and data availability rather than consistent extraction failure. Manual inspection confirmed the agent's inferred full country names and detailed industry classifications often provided superior intelligence value compared to abbreviated codes or sparse reference labels. Play's 89.15% country accuracy demonstrates that when country names

appear explicitly in structured formats, the agent achieves high semantic similarity scores, whilst lower scores for other groups reflect format mismatches between full names and ISO codes rather than extraction errors.

However, claiming the agent demonstrates "superior semantic understanding" conflates two distinct issues. First, the evaluation metric fails to recognise valid representational differences, which is a methodological problem requiring stratified metrics that distinguish exact-match fields from inference fields. Second, whether inferred values provide "more useful intelligence" depends on downstream application requirements that remain unspecified. Threat intelligence analysts may prefer ISO codes for database consistency, whilst full country names benefit human readability. The evaluation cannot determine which representation has greater utility without defining specific analytical objectives.

Future frameworks should implement stratified evaluation that distinguishes between literal extraction and semantic inference. For instance, LLM-based evaluators have demonstrated effectiveness in assessing natural language generation quality [50] and can serve as automated judges for complex evaluation tasks [51]. These evaluators could assess semantic equivalence for fields like geography and industry, accepting multiple valid forms whilst verifying that classifications align with established taxonomic standards such as NAICS or GICS codes. Additionally, deep learning approaches to entity matching [52] provide techniques for identifying semantically equivalent representations across heterogeneous data sources. Evaluation must establish ground truth that reflects the system's intended use. The current evaluation approach compares extracted data against ransomware.live as a reference parser, rather than assessing whether the extracted information is actually fit for intelligence analysis purposes. This design fundamentally limits evaluation validity regardless of which similarity metric is chosen, as it measures consistency with another parser's outputs rather than measuring utility for the task at hand.

8. Closing Remarks

Ransomware.agent demonstrates the successful application of a novel AI-powered solution to address a clear limitation in the current industry leading processes for ransomware intelligence collection. By targeting the manual overhead and brittleness of conventional parsing approaches, this work provides evidence that autonomous agents can contribute meaningfully to operational cyber threat intelligence workflows. Whilst the evaluation revealed specific architectural constraints requiring further development, the fundamental approach validates that agentic driven extraction can approach conventional parser performance under appropriate conditions. As ransomware operations continue to evolve their digital infrastructure and adversarial groups maintain their relentless pace of innovation, the cyber intelligence community must leverage emerging technologies to sustain analytical capabilities. This research represents one contribution to equipping defenders with the adaptive tools necessary to keep pace with adversaries that show no signs of slowing.

9. References

[1] "Top Ransomware Statistics and Recent Ransomware Attacks 2025", *Exabeam*, 2025. [Online]. Available: https://www.exabeam.com/explainers/information-security/top-ransomware-statistics-and-recent-ransomware-attacks-2025/ (accessed Nov. 8, 2025).

[2] "Ransomware Statistics by Year: 2025 Comprehensive Report", *Total Assure*, 2025. [Online]. Available: https://www.totalassure.com/blog/ransomware-statistics-by-year-2025-comprehensive-report (accessed Nov. 8, 2025).

[3] Security Magazine Staff, "Ransomware Statistics: Updates on Ransoms, Attacks and Active Groups", *Security Magazine*, 2025. [Online]. Available: https://www.securitymagazine.com/articles/101795-ransomware-statistics-updates-on-ransoms-attacks-and-active-groups (accessed Nov. 8, 2025).

[4] "Worldwide Ransomware 2024", *Office of the Director of National Intelligence*, Cyber Threat Intelligence Integration Center, 2024. [Online]. Available: https://www.dni.gov/files/CTIIC/documents/products/Worldwide_Ransomware_2024.pdf (accessed Nov. 8, 2025).

[5] J. Martin, C. Whelan, "Ransomware through the lens of state crime: Conceptualizing ransomware groups as cyber proxies, pirates, and privateers", *State Crime Journal*, vol. 12, no. 1, pp. 5, May 2023. doi: 10.13169/statecrime.12.1.0004.

[6] R. Richardson, M. North, "Ransomware: Evolution, Mitigation and Prevention", *Kennesaw.edu*, 2017. [Online]. Available: https://digitalcommons.kennesaw.edu/facpubs/4276/.

[7] H. Alshaikh, "Ransomware Prevention and Mitigation Techniques", *International Journal of Computer Applications*, Feb. 2020.

[8] H. A. Jawaheri, M. A. Sabah, Y. Boshmaf, A. Erbad, "Deanonymizing Tor hidden service users through Bitcoin transactions analysis", *Computers & Security*, vol. 89, pp. 101684, Feb. 2020. doi: 10.1016/j.cose.2019.101684.

[9] "#StopRansomware: Rhysida Ransomware", *Cybersecurity and Infrastructure Security Agency*, Nov. 15, 2023. [Online]. Available: https://www.cisa.gov/news-events/cybersecurity-advisories/aa23-319a (accessed Mar. 26, 2025).

[10] S. Davidoff, M. Durrin, K. Sprenger, *Ransomware and Cyber Extortion*, Addison-Wesley Professional, 2022.

[11] J. Highet, "ransomwatch", *ransomwatch*, 2021. [Online]. Available: https://ransomwatch.telemetry.ltd/#/ (accessed Mar. 26, 2025).

[12] "Ransomlook", *Ransomlook*, 2025. [Online]. Available: https://www.ransomlook.io (accessed Apr. 19, 2025).

[13] J. Mousqueton, "Ransomware.live", *Ransomware.live*, 2024. [Online]. Available: https://www.ransomware.live/about (accessed Mar. 26, 2025).

[14] A. Dalvi, S. Bhirud, "Dark Web Monitoring as an Emerging Cybersecurity Strategy for Businesses", *International Journal of Information Engineering and Electronic Business*, vol. 16, no. 2, pp. 54–67, Jun. 2024. doi: 10.5815/ijieeb.2024.02.05.

[15] E. Ferrara, P. De Meo, G. Fiumara, R. Baumgartner, "Web data extraction, applications and techniques: A survey", *Knowledge-Based Systems*, vol. 70, pp. 301–323, Nov. 2014. doi: 10.1016/j.knosys.2014.07.007.

[16] "Captive", *Oxford English Dictionary*, Oxford University Press, 2025. [Online]. Available: https://www.oed.com/ (accessed Apr. 20, 2025).

[17] L. Caviglione, et al., "Tight Arms Race: Overview of Current Malware Threats and Trends in Their Detection", *IEEE Access*, vol. 9, pp. 5371–5396, 2021. doi: 10.1109/ACCESS.2020.3048319.

[18] M. Keshavarzi, H. R. Ghaffary, "I2CE3: A dedicated and separated attack chain for ransomware offenses as the most infamous cyber extortion", *Computer Science Review*, vol. 36, pp. 100233, May 2020. doi: 10.1016/j.cosrev.2020.100233.

[19] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System", 2008. [Online]. Available: https://bitcoin.org/bitcoin.pdf.

[20] R. Dingledine, N. Mathewson, P. Syverson, "Tor: The Second-Generation Onion Router", *Proceedings of the 13th USENIX Security Symposium*, Aug. 2004. [Online]. Available: https://svn.torproject.org/svn/projects/design-paper/tor-design.pdf (accessed Apr. 20, 2025).

[21] T. Wang, I. Goldberg, "On Realistically Attacking Tor with Website Fingerprinting", *Proceedings on Privacy Enhancing Technologies*, vol. 2016, no. 4, pp. 21–36, 2016. [Online]. Available: https://petsymposium.org/popets/2016/popets-2016-0027.pdf.

[22] "Tor Exit Nodes: How Attackers Exploit Them and How to Defend", *Malware Patrol - Intelligent Threat Data*, Jun. 25, 2025. [Online]. Available: https://www.malwarepatrol.net/tor-exit-nodes-risks-monitoring-and-defensive-use/ (accessed Apr. 20, 2025).

[23] "Avoid infection by dangerous Onion ransomware aka CTB-Locker", *Kaspersky official blog*, Jan. 28, 2015. [Online]. Available: https://www.kaspersky.com/blog/ctb-locker-onion-ransomware/7884/ (accessed Apr. 20, 2025).

[24] D. Balaban, "Ransomware evolution: From lockers to data leaks", *IT Governance USA Blog*, Aug. 13, 2020. [Online]. Available: https://www.itgovernanceusa.com/blog/ransomware-evolution-from-lockers-to-data-leaks (accessed Apr. 20, 2025).

[25] J. Ispahany, M. R. Islam, M. Z. Islam, M. A. Khan, "Ransomware detection using machine learning: A review, research limitations and future directions", *IEEE Access*, vol. 12, no. 1, pp. 5, Jan. 2024. doi: 10.1109/ACCESS.2024.3397921.

[26] A. Price, "Data-leak site emergence continues to increase", *CYJAX*, Aug. 19, 2024. [Online]. Available: https://www.cyjax.com/resources/blog/data-leak-site-emergence-continues-to-increase/ (accessed Apr. 16, 2025).

[27] M. Khder, "Web Scraping with Regular Expressions and Wrappers", *Turkish Online Journal of Science & Technology*, vol. 11, no. 1, pp. 51–58, Jan. 2018. [Online]. Available: https://www.tojsat.net/journals/tojdel/articles/v11i01c02/v11i01-51.pdf (accessed Apr. 20, 2025).

[28] "Web Scraping History: The Origins of Web Scraping", *Scraping Robot*, Mar. 1, 2022. [Online]. Available: https://scrapingrobot.com/blog/web-scraping-history/ (accessed Apr. 20, 2025).

[29] "What Is Web Scraping?", *ScrapingBee Blog*, Feb. 8, 2021. [Online]. Available: https://www.scrapingbee.com/blog/what-is-web-scraping/ (accessed Apr. 20, 2025).

[30] R. Gur et al., "Understanding HTML with Large Language Models", *arXiv*, May 19, 2023. [Online]. Available: https://arxiv.org/abs/2210.03945.

[31] V. Clairoux-Trepanier, I.-M. Beauchamp, E. Ruellan, M. Paquet-Clouston, S.-O. Paquette, E. Clay, "The Use of Large Language Models (LLM) for Cyber Threat Intelligence (CTI) in Cybercrime Forums", *arXiv*, Oct. 1, 2024. [Online]. Available: https://arxiv.org/pdf/2408.03354 (accessed Mar. 26, 2025).

[32] Z. Naidu, et al., "A Whole New World: Creating a Parallel-Poisoned Web Only AI-Agents Can See", *arXiv*, Sep. 2025. [Online]. Available: https://arxiv.org/html/2509.00124v1.

[33] "What Are Agentic Workflows?", *IBM Think Blog*, Aug. 2024. [Online]. Available: https://www.ibm.com/think/topics/agentic-workflows (accessed Apr. 20, 2025).

[34] T. Abuelsaad, et al., "Agent-E: From Autonomous Web Navigation to Foundational Design Principles in Agentic Systems", Jul. 2024. [Online]. Available: https://arxiv.org/html/2407.13032v1.

[35] "Playwright Documentation", *Playwright.dev*, Microsoft, 2025. [Online]. Available: https://playwright.dev/ (accessed Oct. 29, 2025).

[36] "MongoDB Documentation", *MongoDB*, MongoDB, Inc., 2025. [Online]. Available: https://www.mongodb.com/docs/ (accessed Nov. 3, 2025).

[37] "DeepSeek API Documentation", *DeepSeek*, 2025. [Online]. Available: https://api-docs.deepseek.com/ (accessed Nov. 1, 2025).

[38] G. Salton, M. J. McGill, *Introduction to Modern Information Retrieval*, New York: McGraw-Hill, 1983.

[39] A. Singhal, "Modern Information Retrieval: A Brief Overview", *IEEE Data Engineering Bulletin*, vol. 24, no. 4, pp. 35–43, 2001. [Online]. Available: https://www.csee.umbc.edu/courses/graduate/676/fall14/01/papers/singhal_2001.pdf.

[40] N. Reimers, I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks", *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, Nov. 2019. doi: 10.18653/v1/D19-1410. [Online]. Available: https://arxiv.org/abs/1908.10084.

[41] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, M. Zhou, "MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers", *Advances in Neural*

Information Processing Systems, vol. 33, 2020. [Online]. Available: https://arxiv.org/abs/2002.10957.

[42] "Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context", *Google AI Blog*, Feb. 2024. [Online]. Available: https://blog.google/technology/ai/google-gemini-next-generation-model-february-2024/ (accessed Nov. 2, 2025).

[43] S. Zhou, et al., "WebArena: A Realistic Web Environment for Building Autonomous Agents", *arXiv*, 2023. [Online]. Available: https://arxiv.org/abs/2307.13854.

[44] X. Deng, et al., "Mind2Web: Towards a Generalist Agent for the Web", *Advances in Neural Information Processing Systems*, 2023. [Online]. Available: https://arxiv.org/abs/2306.06070.

[45] "Retrieval-augmented generation", *Wikipedia*, 2024. [Online]. Available: https://en.wikipedia.org/wiki/Retrieval-augmented_generation (accessed Nov. 2, 2025).

[46] H. Chen, et al., "VDocRAG: A Unified Framework for Visually Rich Document Retrieval-Augmented Generation", *arXiv*, Apr. 2025. [Online]. Available: https://arxiv.org/abs/2504.09795.

[47] Y. Liu, et al., "ECoRAG: Evidence-based Compression for Retrieval Augmented Generation", *arXiv*, Jun. 2025. [Online]. Available: https://arxiv.org/abs/2506.05167.

[48] "Introducing Perplexity Comet", *Perplexity AI*, 2025. [Online]. Available: https://www.perplexity.ai/comet (accessed Nov. 2, 2025).

[49] "Introducing ChatGPT Atlas", *OpenAI*, 2025. [Online]. Available: https://openai.com/index/introducing-chatgpt-atlas/ (accessed Nov. 2, 2025).

[50] Y. Liu, et al., "G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment", *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. [Online]. Available: https://arxiv.org/abs/2303.16634.

[51] L. Zheng, et al., "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena", *Advances in Neural Information Processing Systems*, 2023. [Online]. Available: https://arxiv.org/abs/2306.05685.

[52] S. Mudgal, et al., "Deep Learning for Entity Matching: A Design Space Exploration", *Proceedings of the 2018 International Conference on Management of Data*, 2018. doi: 10.1145/3183713.3196926.

[53] R. L. Rivest, A. Shamir, L. Adleman, "A method for obtaining digital signatures and public-key cryptosystems", *Communications of the ACM*, vol. 21, no. 2, pp. 120–126, Feb. 1978. doi: 10.1145/359340.359342.

[54] J. Hong, "The State of Phishing Attacks", *Communications of the ACM*, vol. 55, no. 1, pp. 74–81, Jan. 2012. doi: 10.1145/2063176.2063197.

[55] "#StopRansomware: LockBit 3.0 Ransomware Affiliates Exploit CVE 2023-4966 Citrix Bleed Vulnerability", *Cybersecurity and Infrastructure Security Agency*, Nov. 8, 2023. [Online]. Available: https://www.cisa.gov/news-events/cybersecurity-advisories/aa23-325a (accessed Nov. 2, 2025).

[56] "Akira Ransomware", *Arctic Wolf*, 2024. [Online]. Available: https://arcticwolf.com/resources/blog/akira-ransomware/ (accessed Nov. 2, 2025).

[57] L. Richardson, "Beautiful Soup Documentation", 2004-2025. [Online]. Available: https://www.crummy.com/software/BeautifulSoup/ (accessed Nov. 2, 2025).

[58] R. A. Brooks, "A Robust Layered Control System for a Mobile Robot", *IEEE Journal on Robotics and Automation*, vol. 2, no. 1, pp. 14–23, Mar. 1986. doi: 10.1109/JRA.1986.1087032.

[59] M. Albarracin, et al., "From Sensing to Action: A Comparative Analysis of Perception and Behavior in Embodied AI Systems", *arXiv*, Jul. 2024. [Online]. Available: https://arxiv.org/abs/2407.18651.

10. Appendix

10.1. Abbreviations and Acronyms

Abbreviation	Definition
AI	Artificial Intelligence
API	Application Programming Interface
CLI	Command-Line Interface
DLS	Data Leak Sites [9]
DOM	Document Object Model
GICS	Global Industry Classification Standard
HTML	HyperText Markup Language
ISO	International Organization for Standardization
JSON	JavaScript Object Notation
LLM	Large Language Model
NAICS	North American Industry Classification System
RaaS	Ransomware as a Service
RAG	Retrieval-Augmented Generation [45]
TOR	The Onion Router [20]
URL	Uniform Resource Locator

10.2. Source Code Repository

The complete source code for *Ransomware.agent* is publicly available on GitHub under the MIT License:

`https://github.com/kk4w4i/ransomware.agent`

The repository contains the API implementation, action library, and system architecture described in this thesis. As noted in the repository documentation, the system requires access to a configured MongoDB database and is designed as a backend service rather than a standalone end-user application.